Optimization-Based Clinical Decision Support for Curable Hypertension

SVM: Hard-Margin Feasibility, Soft Margins, and Kernel Validation

Nguyen Thanh Tung

Optimization for Data Science Project

Singapore University of Technology and Design

July 16, 2026

1 Problem Identification

Primary aldosteronism (PA) is an important cause of secondary hypertension. The clinically relevant subtype decision is whether PA is unilateral or bilateral. Unilateral PA can often be treated surgically, whereas bilateral PA is usually managed medically [3]. The aim of this project is to formulate this binary classification problem as a maximum-margin optimization problem and to determine whether a linear hard-margin support vector machine (SVM) can be trained on the available project data.

The analysis begins with a *linear hard-margin* SVM to determine whether a perfect linear separator exists. That feasibility audit uses no slack variables, finite penalty parameter, hinge-loss penalty, or kernel transformation. Once the audit proves that the classes are not linearly separable, the report introduces a linear soft-margin SVM, derives its primal and dual formulations, and selects its penalty parameter C by leakage-safe grid search. The resulting sequence is therefore hard-margin feasibility, soft-margin optimization, nested cross-validated performance, and an RBF kernel comparison with explicit overfitting diagnostics.

2 Dataset and Feature Representation

The source report describes a cohort of 114 patients evaluated for PA at Changi General Hospital [4]. Patient-level clinical data were not available for this course project, so the numerical analysis uses a synthetic CGH-like cohort with 114 observations: 56 labelled unilateral PA (UPA) and 58 labelled bilateral PA (BPA). The synthetic data are used to test the optimization workflow and are not evidence of clinical performance.

Ten continuous predictors are used: plasma aldosterone concentration, plasma renin activity, potassium, tumor size, 18-hydroxycortisol, 18-oxocortisol, systolic blood pressure, diastolic blood pressure, antihypertensive burden, and age. Let $x_i \in \mathbb{R}^p$ denote the feature vector for observation i, with $p = 10$, and let

$$t_i = \begin{cases} +1, & \text{for UPA,} \\ -1, & \text{for BPA.} \end{cases} \tag{2.1}$$

The four positively skewed hormone measurements are log-transformed. Within each cross-validation split, medians, means, and standard deviations are estimated from the training fold only; these training-fold quantities are then used for imputation and standardization of the held-out fold.

3 Hard-Margin SVM: Primal, Dual, and Existence

3.1 Primal Optimization Problem

The linear decision score is $s_i = w^\top x_i + b$, where $w \in \mathbb{R}^p$ is the normal vector of the separating hyperplane and $b \in \mathbb{R}$ is the intercept. The hard-margin primal SVM is the convex quadratic program [1]

$$\begin{aligned} \min_{w,b} \quad & \phi(w,b) = \frac{1}{2}\|w\|_2^2 \\ \text{subject to} \quad & t_i(w^\top x_i + b) \geq 1, \qquad i = 1, \ldots, m. \end{aligned} \tag{3.1}$$

The constraints require every training observation to be correctly classified with functional margin at least one. Minimizing $\|w\|_2^2/2$ maximizes the geometric margin $2/\|w\|_2$ between the two supporting hyperplanes.

In a hard-margin SVM there is no per-observation loss term to differentiate. Margin violations are forbidden by constraints rather than assigned a finite loss. Consequently, the quantity differentiated below is the *primal objective*, together with the affine constraint functions.

3.2 First and Second Derivatives

Define the augmented parameter vector, quadratic matrix, and signed augmented feature vector as

$$\theta = \begin{bmatrix} w \\ b \end{bmatrix}, \qquad Q = \begin{bmatrix} I_p & 0 \\ 0 & 0 \end{bmatrix}, \qquad u_i = t_i \begin{bmatrix} x_i \\ 1 \end{bmatrix}. \tag{3.2}$$

The objective and constraints can then be written as

$$\phi(\theta) = \frac{1}{2}\theta^\top Q\theta, \qquad g_i(\theta) = 1 - u_i^\top \theta \le 0. \tag{3.3}$$

The first derivative of the objective is

$$\nabla\phi(\theta) = Q\theta = \begin{bmatrix} w \\ 0 \end{bmatrix}, \tag{3.4}$$

and its second derivative is the constant Hessian

$$\nabla^2\phi(\theta) = Q = \begin{bmatrix} I_p & 0 \\ 0 & 0 \end{bmatrix}. \tag{3.5}$$

For each margin constraint,

$$\nabla g_i(\theta) = -u_i = -t_i \begin{bmatrix} x_i \\ 1 \end{bmatrix}, \qquad \nabla^2 g_i(\theta) = 0. \tag{3.6}$$

Because $Q \succeq 0$ and every g_i is affine, the primal problem is convex. The objective is strictly convex in w but not in the complete vector (w, b) because the intercept is unpenalized. Thus the optimal w is unique whenever a solution exists, whereas the intercept need not be unique. There is also no hinge boundary or nondifferentiable point in this constrained primal formulation.

Section 5.2 evaluates $u_i^\top \theta$ row by row in a ten-observation spreadsheet illustration.

3.3 Lagrangian and KKT Conditions

The derivatives of the primal objective describe only how $\|w\|_2^2/2$ changes; they do not by themselves enforce the margin constraints. Indeed, setting the objective gradient to zero would give $w = 0$, which is generally infeasible. The Lagrangian is therefore introduced as an auxiliary function—not as a replacement for the primal objective—to combine the objective with the constraints and express constrained optimality through the KKT conditions.

Introduce one multiplier $\alpha_i \ge 0$ for each inequality constraint. The Lagrangian is

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2}\|w\|_2^2 + \sum_{i=1}^{m} \alpha_i \left[1 - t_i(w^\top x_i + b)\right]. \tag{3.7}$$

Its first derivatives with respect to the primal variables are

$$\nabla_w \mathcal{L} = w - \sum_{i=1}^{m} \alpha_i t_i x_i, \qquad \frac{\partial \mathcal{L}}{\partial b} = -\sum_{i=1}^{m} \alpha_i t_i. \tag{3.8}$$

At a stationary point these derivatives vanish, giving $w = \sum_i \alpha_i t_i x_i$ and $\sum_i \alpha_i t_i = 0$. Thus the separating direction is determined by the training observations whose constraints are active and whose multipliers are positive, rather than by the unconstrained minimum $w = 0$.

For fixed α, the Hessian with respect to (w, b) is again

$$\nabla^2_{(w,b)} \mathcal{L} = \begin{bmatrix} I_p & 0 \\ 0 & 0 \end{bmatrix}. \tag{3.9}$$

When the primal is feasible, its Karush–Kuhn–Tucker (KKT) conditions are necessary and sufficient for global optimality [2]:

$$\begin{aligned} w - \sum_{i=1}^{m} \alpha_i t_i x_i = 0, \quad \sum_{i=1}^{m} \alpha_i t_i = 0, \\ t_i(w^\top x_i + b) - 1 \geq 0, \qquad \alpha_i \geq 0, \\ \alpha_i \left[t_i(w^\top x_i + b) - 1 \right] = 0, \qquad i = 1, \ldots, m. \end{aligned} \tag{3.10}$$

The final equality is complementary slackness. Observations with $\alpha_i > 0$ lie on a supporting margin and are support vectors. Stationarity alone is not sufficient: primal feasibility, dual feasibility, and complementary slackness must also hold.

The KKT conditions are sufficient here because the objective is convex and the margin constraints are affine. They are also necessary whenever the hard-margin problem is feasible. To see the required strict feasibility, scale any feasible separator (w, b) by a constant $c > 1$; then $t_i((cw)^\top x_i + cb) \geq c > 1$ for every observation, so Slater's condition holds. Conversely, if the data are not linearly separable, primal feasibility fails and no triplet (w, b, α) can satisfy the complete KKT system.

3.4 Dual Problem and the Meaning of the Dual Bound

The primal and dual optimize different variables and therefore point in opposite directions. The primal minimizes over the classifier parameters (w, b). For any fixed multiplier vector $\alpha \geq 0$, define the dual function as the smallest Lagrangian value attainable by the primal variables,

$$q(\alpha) = \inf_{w,b} \mathcal{L}(w, b, \alpha). \tag{3.11}$$

For every primal-feasible (w, b), the constraint terms $1 - t_i(w^\top x_i + b)$ are nonpositive. Hence

$$q(\alpha) \leq \mathcal{L}(w, b, \alpha) \leq \frac{1}{2}\|w\|_2^2. \tag{3.12}$$

Thus each dual-feasible α supplies a *lower bound* on every feasible primal objective value. This is the dual bound. The dual maximizes $q(\alpha)$ because it seeks the largest, and therefore tightest, lower bound; this explains why the primal is a minimization problem while the dual is a maximization problem.

Using stationarity, $w = \sum_i \alpha_i t_i x_i$ and $\sum_i \alpha_i t_i = 0$, eliminates w and b from the Lagrangian and gives

$$\begin{aligned} \max_{\alpha} \quad & q(\alpha) = \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j t_i t_j x_i^\top x_j \\ \text{subject to} \quad & \alpha_i \geq 0, \qquad \sum_{i=1}^{m} \alpha_i t_i = 0. \end{aligned} \tag{3.13}$$

Weak duality always gives $d^* \le p^*$. When the hard-margin constraints are feasible, scaling a feasible separator produces strict feasibility, so Slater's condition holds and strong duality gives $d^* = p^*$. If the data are not linearly separable, however, the primal has no feasible value p^* and the primal–dual optimality calculation cannot produce a hard-margin classifier. Feasibility must therefore be checked before interpreting a dual solution as a trained model.

3.5 Linear Separability and Existence of a Hard-Margin Solution

Let $U \in \mathbb{R}^{m\times(p+1)}$ have rows $u_i^\top = t_i[x_i^\top, 1]$ and let $\theta = [w^\top, b]^\top$. A hard-margin solution exists if and only if the linear system

$$U\theta \ge \mathbf{1} \tag{3.14}$$

is feasible. This condition is also equivalent to strict linear separability: if some hyperplane correctly separates all observations with strictly positive signed scores, rescaling (w, b) makes the smallest signed score equal to one; conversely, any point satisfying the unit-margin inequalities is a strict separator.

The existence question is checked by the zero-objective linear program

$$\min_{\theta}\ 0 \qquad \text{subject to} \qquad -U\theta \le -\mathbf{1}. \tag{3.15}$$

If it is feasible, its output supplies a valid starting point for minimizing the quadratic primal objective. If it is infeasible, changing the objective from zero to $\|w\|_2^2/2$ cannot make the empty feasible set nonempty: the hard-margin primal has no solution. This logical order separates two questions that are often confused: feasibility asks whether any separating hyperplane exists, whereas optimization selects the maximum-margin hyperplane only after existence has been established.

3.6 Worked Separable Primal–Lagrangian–Dual Example

This deliberately small example makes the primal, Lagrangian, and dual views visible before returning to the clinical feature space. It uses the five observations in Table 1. Each point is embedded in the same ten-dimensional notation as the main analysis by setting $x_3 = \cdots = x_{10} = 0$. The symbol y_i used in this example has exactly the same role as t_i elsewhere in the report. Numerical values in the table, solution, margin evaluation, multiplier calculation, dual check, and figure are displayed to two decimal places; exact integers are retained where they make the symbolic constraint algebra easier to read.

Table 1: Five-point teaching dataset embedded in ten feature dimensions.

Point	x_1	x_2	$x_3, \ldots, x_{10}$	y
A	−2.00	0.00	0.00	−1
B	−1.00	1.00	0.00	−1
C	1.00	1.00	0.00	+1
D	2.00	0.00	0.00	+1
E	2.00	2.00	0.00	+1

Geometric intuition. The closest opposite-class observations in the horizontal direction are B at $x_1 = -1.00$ and C at $x_1 = 1.00$. The widest symmetric separating strip therefore places its decision boundary halfway between them at $x_1 = 0.00$, with supporting margins at $x_1 = -1.00$ and $x_1 = 1.00$. Figure 1 shows this geometry. B and C touch the margins, while A, D, and E are farther from the decision boundary.

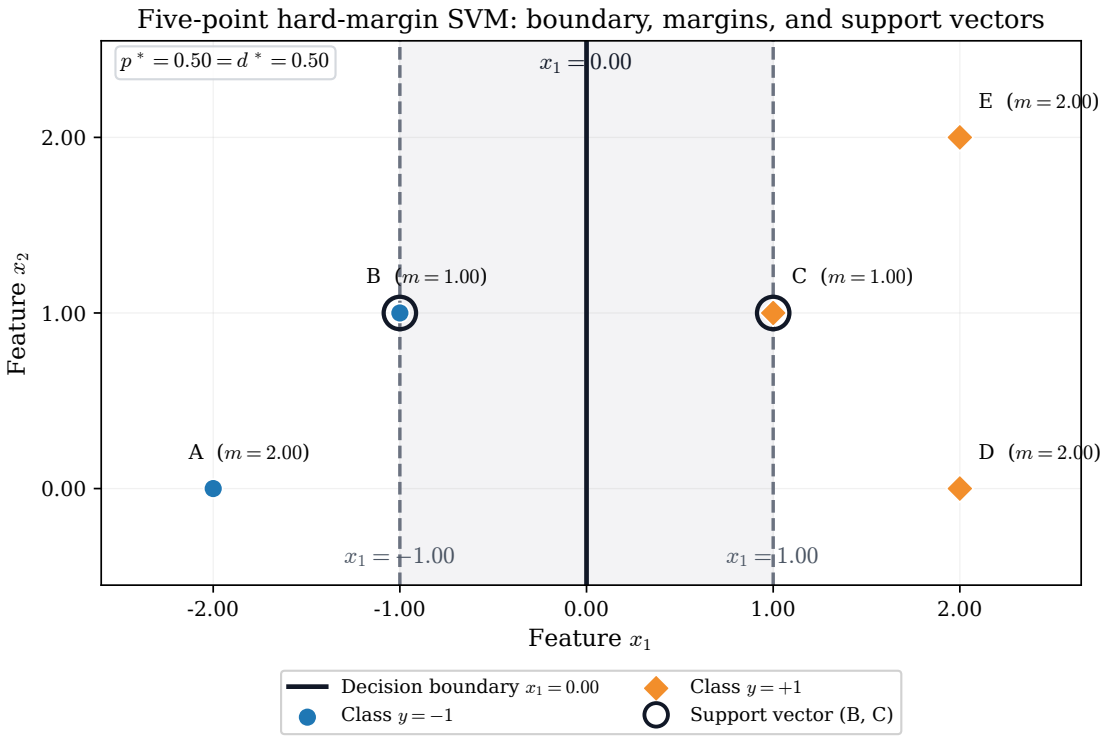

Figure 1: Five-point hard-margin example. The black line is the decision boundary, the dashed lines are the two supporting margins, and the rings identify support vectors B and C. Here $m_i = y_i(w^\top x_i + b)$ denotes the signed functional margin.

Primal problem and the five constraints. For $w = (w_1, \dots, w_{10})^\top$, the hard-margin primal is

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2}\|w\|_2^2 = \frac{1}{2}\sum_{j=1}^{10} w_j^2 \\ \text{subject to} \quad & y_i(w^\top x_i + b) \geq 1, \qquad i \in \{A, B, C, D, E\}. \end{aligned} \tag{3.16}$$

Substituting every observation gives

$$\begin{aligned} \text{A:} \quad & -1[-2w_1 + 0w_2 + 0w_3 + \cdots + 0w_{10} + b] \geq 1 \quad \Longleftrightarrow \quad 2w_1 - b \geq 1, \\ \text{B:} \quad & -1[-w_1 + w_2 + 0w_3 + \cdots + 0w_{10} + b] \geq 1 \quad \Longleftrightarrow \quad w_1 - w_2 - b \geq 1, \\ \text{C:} \quad & +1[w_1 + w_2 + 0w_3 + \cdots + 0w_{10} + b] \geq 1 \quad \Longleftrightarrow \quad w_1 + w_2 + b \geq 1, \\ \text{D:} \quad & +1[2w_1 + 0w_2 + 0w_3 + \cdots + 0w_{10} + b] \geq 1 \quad \Longleftrightarrow \quad 2w_1 + b \geq 1, \\ \text{E:} \quad & +1[2w_1 + 2w_2 + 0w_3 + \cdots + 0w_{10} + b] \geq 1 \quad \Longleftrightarrow \quad 2w_1 + 2w_2 + b \geq 1. \end{aligned} \tag{3.17}$$

This is not an unconstrained optimization problem. If one differentiates only the objective and sets its gradient to zero, then

$$\nabla_w \left(\frac{1}{2}\|w\|_2^2 \right) = w = 0. \tag{3.18}$$

At $w = 0$, the positive-class constraints require $b \geq 1$, whereas the negative-class constraints require $b \leq -1$; no intercept can satisfy both. The unconstrained minimum is therefore infeasible, which is precisely why the margin constraints must enter the optimality analysis through the Lagrangian and KKT conditions.

Solving the primal step by step. Add the simplified constraints for B and C:

$$(w_1 - w_2 - b) + (w_1 + w_2 + b) \geq 2 \quad \Longrightarrow \quad w_1 \geq 1. \tag{3.19}$$

It follows that every feasible point satisfies

$$\frac{1}{2}\|w\|_2^2 = \frac{1}{2}(w_1^2 + w_2^2 + \cdots + w_{10}^2) \geq \frac{1}{2}. \tag{3.20}$$

Choose $w_1 = 1.00$, $w_2 = \cdots = w_{10} = 0.00$. The B constraint then gives $1.00 - b \geq 1.00$, hence $b \leq 0.00$, while the C constraint gives $1.00 + b \geq 1.00$, hence $b \geq 0.00$. Thus $b = 0.00$. This candidate also satisfies the A, D, and E constraints, so it attains the lower bound and is globally optimal:

$$w^* = (1.00, 0.00, \ldots, 0.00)^\top, \qquad b^* = 0.00, \qquad p^* = \frac{1}{2}\|w^*\|_2^2 = 0.50. \tag{3.21}$$

The decision equation $(w^*)^\top x + b^* = 0$ reduces to $x_1 = 0.00$.

Table 2: Signed functional margins at the primal optimum.

Point	y_i	$(w^*)^\top x_i + b^*$	$y_i((w^*)^\top x_i + b^*)$	Status
A	-1	-2.00	2.00	Outside margin
B	-1	-1.00	1.00	Support vector
C	$+1$	1.00	1.00	Support vector
D	$+1$	2.00	2.00	Outside margin
E	$+1$	2.00	2.00	Outside margin

Table 2 confirms that B and C are support vectors because their signed margins equal 1.00. A, D, and E have signed margins strictly greater than 1.00 and therefore lie outside the margin strip.

Lagrangian, stationarity, and complementary slackness. Introduce one multiplier $\alpha_i \geq 0$ for each of the five constraints:

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2}\|w\|_2^2 - \sum_{i \in \{A,B,C,D,E\}} \alpha_i \left[y_i(w^\top x_i + b) - 1 \right]. \tag{3.22}$$

Stationarity with respect to the primal variables gives

$$\begin{aligned} \nabla_w \mathcal{L} = 0 \quad &\Longrightarrow \quad w = \sum_i \alpha_i y_i x_i, \\ \frac{\partial \mathcal{L}}{\partial b} = 0 \quad &\Longrightarrow \quad \sum_i \alpha_i y_i = 0. \end{aligned} \tag{3.23}$$

Complementary slackness requires

$$\alpha_i \left[y_i(w^\top x_i + b) - 1 \right] = 0 \qquad \text{for every } i. \tag{3.24}$$

For A, D, and E the bracket equals $2.00 - 1.00 = 1.00 > 0$, so complementary slackness forces

$$\alpha_A = \alpha_D = \alpha_E = 0.00. \tag{3.25}$$

For B and C the bracket is zero, so their multipliers may be positive. The intercept stationarity equation becomes

$$-\alpha_B + \alpha_C = 0 \quad \Longrightarrow \quad \alpha_B = \alpha_C = \gamma. \tag{3.26}$$

Substituting this equality into the weight stationarity equation gives

$$\begin{aligned} w &= \gamma(-1)(-1, 1, 0, \ldots, 0) + \gamma(+1)(1, 1, 0, \ldots, 0) \\ &= (2\gamma, 0, \ldots, 0). \end{aligned} \tag{3.27}$$

Matching the primal solution $w^* = (1.00, 0.00, \ldots, 0.00)$ gives $2.00\gamma = 1.00$, and therefore

$$\alpha_B = \alpha_C = 0.50. \tag{3.28}$$

In the two visible coordinates, the reconstruction is

$$0.50(-1)(-1.00, 1.00) + 0.50(+1)(1.00, 1.00) = (1.00, 0.00), \tag{3.29}$$

which verifies that the two support vectors determine the separating direction.

Dual problem and strong duality. Minimizing the Lagrangian over w and b using the stationarity equations produces the hard-margin dual

$$\begin{aligned} \max_{\alpha} \quad & q(\alpha) = \sum_i \alpha_i - \frac{1}{2}\left\| \sum_i \alpha_i y_i x_i \right\|_2^2 \\ \text{subject to} \quad & \alpha_i \geq 0, \qquad \sum_i \alpha_i y_i = 0. \end{aligned} \tag{3.30}$$

For this dataset, the two nonzero coordinates of $\sum_i \alpha_i y_i x_i$ are

$$\begin{aligned} z_1 &= 2\alpha_A + \alpha_B + \alpha_C + 2\alpha_D + 2\alpha_E, \\ z_2 &= -\alpha_B + \alpha_C + 2\alpha_E, \end{aligned} \tag{3.31}$$

so the complete dataset-specific dual is

$$\begin{aligned} \max_{\alpha \geq 0} \quad & \alpha_A + \alpha_B + \alpha_C + \alpha_D + \alpha_E \\ & - \frac{1}{2}\left[(2\alpha_A + \alpha_B + \alpha_C + 2\alpha_D + 2\alpha_E)^2 + (-\alpha_B + \alpha_C + 2\alpha_E)^2 \right] \\ \text{subject to} \quad & -\alpha_A - \alpha_B + \alpha_C + \alpha_D + \alpha_E = 0. \end{aligned} \tag{3.32}$$

At $(\alpha_A, \alpha_B, \alpha_C, \alpha_D, \alpha_E) = (0.00, 0.50, 0.50, 0.00, 0.00)$, the equality constraint holds and

$$q(\alpha) = 0.50 + 0.50 - \frac{1}{2}[(0.50 + 0.50)^2 + (-0.50 + 0.50)^2] = 0.50. \tag{3.33}$$

The primal construction already supplies a feasible value $p^* = 0.50$, while this dual-feasible point supplies the lower bound $d^* = 0.50$. Weak duality prevents a dual value from exceeding a primal value, so equality certifies that both points are optimal:

$$\boxed{d^* = 0.50 = p^*}. \tag{3.34}$$

This zero duality gap is the numerical manifestation of strong duality. The figure and all calculations in this subsection are reproduced by `scripts/build_svm_primal_dual_toy_example.py`.

4 Linear Soft-Margin SVM

4.1 Why Slack Variables Are Needed

The hard-margin analysis, with the detailed certificate given in Section 5.1, establishes that no (w, b) can satisfy all 114 unit-margin inequalities simultaneously. The soft-margin model changes the assumption rather than attempting to optimize over an empty set. For each observation it introduces a slack variable $\xi_i \geq 0$ that measures the shortfall from the desired signed functional margin of one:

$$\xi_i = \max\{0, 1 - t_i(w^\top x_i + b)\}. \tag{4.1}$$

When $\xi_i = 0$, the observation lies on or outside its supporting margin. If $0 < \xi_i \leq 1$, it lies inside the margin but remains on the correct side of the decision boundary. If $\xi_i > 1$, its signed score is negative and it is misclassified. This interpretation directly connects the optimization variables to the geometry of overlapping UPA and BPA observations.

4.2 Soft-Margin Primal Problem

The linear soft-margin primal problem is

$$\begin{aligned} \min_{w,b,\xi} \quad & p(w,b,\xi) = \frac{1}{2}\|w\|_2^2 + C\sum_{i=1}^{m}\xi_i \\ \text{subject to} \quad & t_i(w^\top x_i + b) \geq 1 - \xi_i, \qquad i = 1,\ldots,m, \\ & \xi_i \geq 0, \qquad i = 1,\ldots,m. \end{aligned} \tag{4.2}$$

The first term favors a wide margin, while the second penalizes total margin violation. The constant $C > 0$ controls their trade-off. Unlike the hard-margin problem, this primal is always feasible: $w = 0$, $b = 0$, and $\xi_i = 1$ satisfy every constraint. Eliminating the slack variables gives the equivalent unconstrained hinge-loss form

$$\min_{w,b} \quad \frac{1}{2}\|w\|_2^2 + C\sum_{i=1}^{m}\max\{0, 1 - t_i(w^\top x_i + b)\}. \tag{4.3}$$

The constrained form is retained below because it makes the primal–dual relationship and the role of C explicit.

4.3 Lagrangian and KKT Conditions

Introduce $\alpha_i \geq 0$ for $1 - \xi_i - t_i(w^\top x_i + b) \leq 0$ and $\mu_i \geq 0$ for $-\xi_i \leq 0$. The Lagrangian is

$$\begin{aligned} \mathcal{L}(w,b,\xi,\alpha,\mu) = \frac{1}{2}\|w\|_2^2 + C\sum_i \xi_i \\ + \sum_i \alpha_i[1 - \xi_i - t_i(w^\top x_i + b)] - \sum_i \mu_i\xi_i. \end{aligned} \tag{4.4}$$

Stationarity with respect to the primal variables gives

$$w = \sum_i \alpha_i t_i x_i, \qquad \sum_i \alpha_i t_i = 0, \qquad C - \alpha_i - \mu_i = 0. \tag{4.5}$$

Because $\mu_i \geq 0$, the final equality produces the upper bound $0 \leq \alpha_i \leq C$. The complete KKT system additionally requires primal feasibility, dual feasibility, and complementary slackness:

$$\begin{aligned} & t_i(w^\top x_i + b) - 1 + \xi_i \geq 0, \qquad \xi_i \geq 0, \\ & 0 \leq \alpha_i \leq C, \\ & \alpha_i[1 - \xi_i - t_i(w^\top x_i + b)] = 0, \\ & (C - \alpha_i)\xi_i = 0. \end{aligned} \tag{4.6}$$

These conditions are necessary and sufficient because the primal objective is convex, its constraints are affine, and strict feasibility is obtained by choosing sufficiently large positive slacks. They also explain the support vectors. An observation with $\alpha_i = 0$ does not determine w and lies outside the margin. A point with $0 < \alpha_i < C$ lies exactly on a supporting margin. A point with $\alpha_i = C$ may lie inside the margin or be misclassified, so it receives the largest multiplier permitted by the chosen penalty.

4.4 Soft-Margin Dual Problem

Minimizing the Lagrangian over w, b, and ξ using the stationarity conditions yields

$$\begin{aligned} \max_{\alpha} \quad & d(\alpha) = \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j t_i t_j x_i^\top x_j \\ \text{subject to} \quad & 0 \le \alpha_i \le C, \qquad \sum_{i=1}^{m} \alpha_i t_i = 0. \end{aligned} \tag{4.7}$$

The objective has the same form as the hard-margin dual, but the new box constraint $\alpha_i \le C$ is essential. It prevents any single observation from exerting unlimited pressure on the separating hyperplane. Since the soft-margin primal is strictly feasible, strong duality holds and the optimal primal and dual values are equal up to numerical solver tolerance.

4.5 Selecting the Penalty Parameter by Grid Search

A small C places greater emphasis on a wide, strongly regularized margin and accepts more slack. A large C penalizes violations more severely and moves toward hard-margin behavior, which can increase sensitivity to overlapping observations. Because neither extreme is known in advance, C is selected from the half-logarithmic grid

$$\mathcal{C} = \{10^{k/2} : k = -4, -3, \ldots, 4\}. \tag{4.8}$$

Displayed to two decimal places, these values are

$$0.01,\ 0.03,\ 0.10,\ 0.32,\ 1.00,\ 3.16,\ 10.00,\ 31.62,\ 100.00. \tag{4.9}$$

Each candidate is evaluated by mean five-fold stratified validation AUROC. The log transformation, median imputation, standardization, and linear SVM are placed in one pipeline, so every preprocessing parameter is estimated from the corresponding training fold only.

Nested cross-validation separates model selection from performance estimation. In each of five outer folds, an independent five-fold inner grid search chooses C using only the outer training observations. The selected model then scores the untouched outer test fold. Table 3 shows that all five inner searches selected $C = 0.01$.

Table 3: Nested cross-validation for soft-margin penalty selection. AUROC values and C are displayed to two decimal places.

Outer fold	Train	Test	Selected C	Inner AUROC	Outer AUROC
1	91	23	0.01	0.95	0.92
2	91	23	0.01	0.96	0.95
3	91	23	0.01	0.94	0.98
4	91	23	0.01	0.91	0.98
5	92	22	0.01	0.95	0.89

4.6 Soft-Margin Solution on the Project Dataset

The final full-data grid search also selected $C = 0.01$, with mean validation AUROC 0.95. Figure 2 shows both the validation curve over C and the nested out-of-fold ROC curve. The nested out-of-fold AUROC is 0.95, with accuracy, sensitivity, and specificity all approximately 0.86 at decision threshold zero. Unlike the earlier squared-hinge result, these numbers are generated by the explicit linear soft-margin primal–dual model defined in this section.

The fitted standardized-space coefficients in Table 4 give the final linear scoring function. Their signs describe the direction of the score after the stated preprocessing but should not be interpreted as causal clinical effects.

Table 4: Final standardized-space soft-margin coefficients at $C = 0.01$.

Parameter	Estimate	Parameter	Estimate
PAC	0.29	18-oxoF	0.22
PRA	-0.18	Systolic BP	0.09
Potassium	-0.19	Diastolic BP	0.18
Tumor size	0.15	DDD	0.07
18-OHF	0.23	Age	-0.08
Intercept	-0.02		

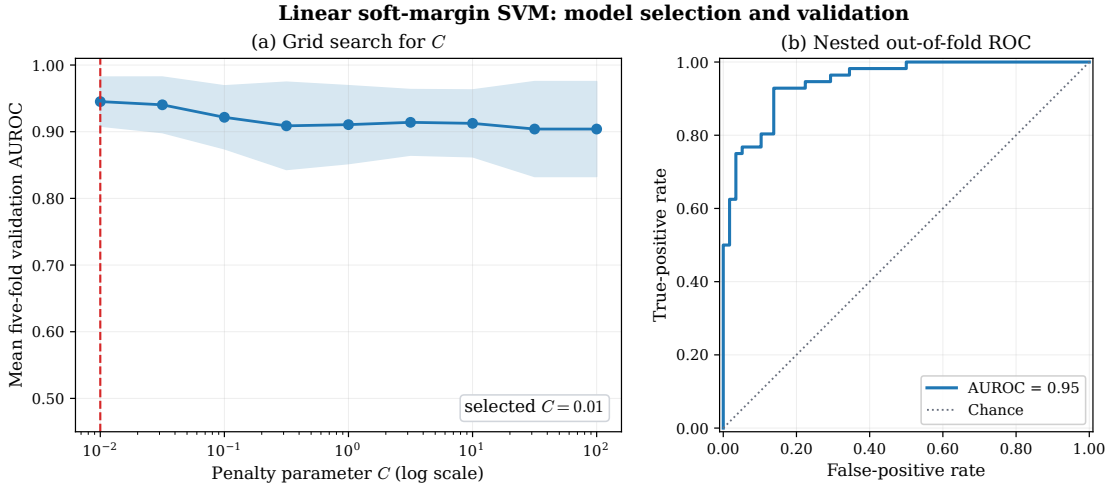

Figure 2: Soft-margin model selection and validation. Panel (a) reports mean five-fold validation AUROC with one standard deviation across folds for every candidate C; the dashed line marks the selected value. Panel (b) reports the nested out-of-fold ROC obtained when C is selected inside each outer training fold.

The full-data solution contains 82 support vectors and 77 observations with positive slack, with $\sum_i \xi_i = 46.89$. Substitution into the primal gives

$$\begin{aligned} p^* &= \frac{1}{2}\|w\|_2^2 + C\sum_i \xi_i \\ &= \frac{1}{2}(0.33) + (0.01)(46.89) \\ &\approx 0.63. \end{aligned} \tag{4.10}$$

The dual calculation gives $d^* \approx 0.63$, and the full-precision primal–dual gap is 4.85×10^{-10}. The remaining numerical checks are $|\sum_i \alpha_i t_i| = 2.78 \times 10^{-17}$ and a zero weight-reconstruction residual at stored precision. The two maximum complementary-slackness residuals are 2.35×10^{-10} and 8.57×10^{-11}, confirming the KKT conditions to solver tolerance. Table 5 summarizes the model-selection and optimization results.

Table 5: Soft-margin model-selection, validation, and optimization summary.

Quantity	Value
Selected C from full-data grid search	0.01
Mean inner-CV AUROC at selected C	0.95
Nested out-of-fold AUROC	0.95
Nested out-of-fold accuracy	0.86
Nested sensitivity	0.86
Nested specificity	0.86
Full-data support vectors	82
Full-data observations with $\xi_i > 0$	77
Full-data $\sum_i \xi_i$	46.89
Full-data primal objective	0.63
Full-data dual objective	0.63

All full-precision grid-search results, nested predictions, coefficients, support-vector multipliers, slacks, and primal–dual checks are written under `output/csv/` by `scripts/linear_soft_margin_svm.py`. Tables in this section and both panels of Figure 2 are generated from those Python outputs.

5 Detailed Hard-Margin Numerical Certificate

5.1 Full-Data Infeasibility Certificate

Three mutually reinforcing checks are used on the complete processed dataset, in the order direct solver status, algebraic Farkas certificate, and geometric convex-hull interpretation. Here $U \in \mathbb{R}^{114\times 11}$ contains the ten standardized features and the intercept column.

Homework-style direct solver verification. The primal model supplied to a constrained solver is

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2}\|w\|_2^2 \\ \text{subject to} \quad & U\theta \geq \mathbf{1}. \end{aligned} \tag{5.1}$$

Before an objective can be minimized, the solver must find a point satisfying all 114 constraints. The implementation therefore first solves the equivalent feasibility model $\min_\theta 0$ subject to $-U\theta \leq -\mathbf{1}$; changing the objective cannot make an empty feasible set nonempty. The direct solver audit returned the result in Table 6.

Table 6: Direct full-data primal feasibility check.

Item	Solver result
Decision variables $(w_1, \ldots, w_{10}, b)$	11
Hard-margin constraints	114
Solver	SciPy HiGHS
HiGHS model status	Status 8: Infeasible
Primal parameters (w, b) returned	None
Primal objective $\frac{1}{2}\|w\|_2^2$	Not defined

Thus the correct statement is that the solver returns an *infeasible status*; it does not return an "infeasible solution." No primal solution exists for the solver to report. This is the direct computational verification used in the homework-style workflow.

Independent certificate. The solver status is then strengthened by a direct, independently checkable proof. Farkas' lemma states that the system $U\theta \geq \mathbf{1}$ is infeasible if there is a vector $\lambda \in \mathbb{R}^{114}$ satisfying

$$\lambda \geq 0, \qquad U^\top \lambda = 0, \qquad \mathbf{1}^\top \lambda = 1. \tag{5.2}$$

The auxiliary linear program for this certificate returned twelve nonzero weights, shown in Table 7; the other 102 weights are zero. Displayed weights use two decimal places as requested. Every check uses the unrounded values in the companion CSV file, which also contains a separate two-decimal display column; the displayed values alone should not be used to recompute the certificate.

Table 7: Nonzero weights in the full-data Farkas infeasibility certificate.

Patient	Class	λ_i	Patient	Class	λ_i
CGH-021	UPA	0.21	CGH-072	BPA	0.04
CGH-024	BPA	0.05	CGH-081	UPA	0.01
CGH-026	BPA	0.16	CGH-092	BPA	0.01
CGH-053	UPA	0.04	CGH-105	UPA	0.01
CGH-058	UPA	0.17	CGH-108	BPA	0.05
CGH-060	BPA	0.18	CGH-111	UPA	0.07

The full-precision numerical audit gives

$$\begin{aligned} \mathbf{1}^\top \lambda &= 1.00, \\ \sum_{i:y_i=+1} \lambda_i &= 0.50, \\ \sum_{i:y_i=-1} \lambda_i &= 0.50, \\ \|U^\top \lambda\|_\infty &= 2.50 \times 10^{-16}. \end{aligned} \tag{5.3}$$

The first three quantities are displayed to two decimal places. The final residual is retained in scientific notation because rounding it to 0.00 would hide the numerical accuracy of the certificate. Now suppose, for contradiction, that a hard-margin parameter θ satisfied $U\theta \geq \mathbf{1}$. Nonnegativity of λ would imply

$$\underbrace{(U^\top \lambda)^\top \theta}_{0} = \lambda^\top U\theta \geq \lambda^\top \mathbf{1} = 1, \tag{5.4}$$

which is the contradiction $0 \geq 1$. Therefore the complete hard-margin constraint system has no feasible solution. In fact, the twelve observations in Table 7 already provide a sufficient infeasible subsystem.

The same certificate has a geometric interpretation. Because the intercept component of $U^\top \lambda = 0$ gives equal class masses of 0.5, define $\beta_i^+ = 2\lambda_i$ for UPA and $\beta_i^- = 2\lambda_i$ for BPA. Each set of coefficients is nonnegative and sums to one, while the feature components of $U^\top \lambda = 0$ give

$$\sum_{i:y_i=+1} \beta_i^+ x_i = \sum_{i:y_i=-1} \beta_i^- x_i. \tag{5.5}$$

Thus the standardized UPA and BPA convex hulls contain the same point. Their largest coordinate difference is only 6.66×10^{-16}, as illustrated in Figure 3. Intersecting convex hulls cannot be strictly separated by a hyperplane.

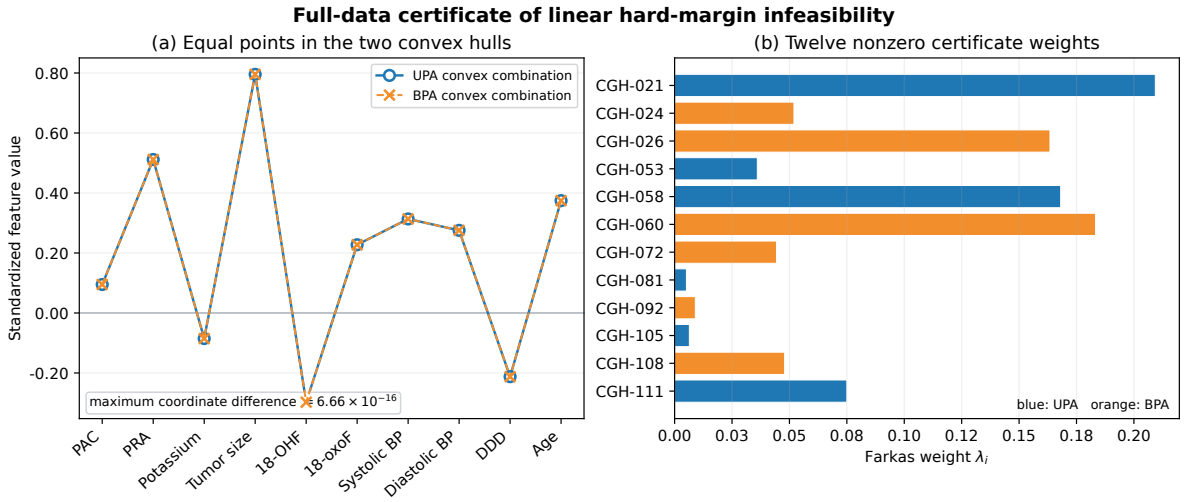

Figure 3: Full-data infeasibility certificate. Panel (a) shows the identical ten-dimensional point obtained as a convex combination of UPA observations and as a convex combination of BPA observations; the plot compares coordinates directly and is not a two-dimensional projection. Panel (b) shows the twelve nonzero Farkas weights.

Consequently, for the current preprocessed project data,

$$\boxed{\text{the UPA and BPA classes are not linearly separable.}} \tag{5.6}$$

The certificate, CSV output, numerical checks, and figure are reproduced by `scripts/build_full_data_infeasibility_certificate.py`. The script writes four machine-readable audit files under `output/csv/`: the direct solver status, sparse Farkas vector, two convex-hull representations, and numerical residual checks. The two tables above are generated from the same Python results and then included in this PDF.

5.2 Supplementary Ten-Observation Spreadsheet Illustration

To make the matrix constraint concrete, a self-contained teaching table was constructed from the first ten observations in the project spreadsheet. The selection rule is transparent: scan from top to bottom, take the first five UPA and first five BPA observations, and restore source-row order. In this dataset these are exactly rows 2–11. The four hormone variables are log-transformed and all ten features are standardized using statistics from these ten rows. No selected value is missing, so median imputation is inactive.

For $p = 10$, the vectors used in one constraint row are

$$\theta = \begin{bmatrix} w_1 & \cdots & w_{10} & b \end{bmatrix}^\top, \qquad u_i = t_i \begin{bmatrix} x_{i1} & \cdots & x_{i,10} & 1 \end{bmatrix}^\top. \tag{5.7}$$

Their inner product is the signed functional margin:

$$u_i^\top \theta = t_i(x_i^\top w + b) = t_i s_i. \tag{5.8}$$

The expression is therefore not "$u_i > 1$": u_i is a vector. The scalar dot product $u_i^\top \theta$ must satisfy $u_i^\top \theta \geq 1$. For UPA, $t_i = +1$ and the rule becomes $s_i \geq 1$; for BPA, $t_i = -1$ and it becomes $s_i \leq -1$.

Solving the primal quadratic program for this teaching subsystem gives

$$\begin{aligned} w_{\text{teach}}^\top &\approx (0.34, -0.42, -0.65, 0.08, -0.10, \\ &\qquad 0.16, -0.13, 0.39, -0.75, 0.64), \\ b_{\text{teach}} &\approx 0.29. \end{aligned} \tag{5.9}$$

These coefficients are displayed to two decimal places; all constraint and objective checks use the full-precision values stored in the generated CSV files.

Numerical evaluation of the objective. The ten observations contribute ten margin constraints; they do not create ten additive loss terms. Thus the fitted teaching subsystem has one global primal objective. Substituting the fitted weights gives

$$\begin{aligned}
\|w_{\text{teach}}\|_2^2 &\approx (0.34)^2 + (-0.42)^2 + (-0.65)^2 + (0.08)^2 + (-0.10)^2 \\
&\quad + (0.16)^2 + (-0.13)^2 + (0.39)^2 + (-0.75)^2 + (0.64)^2 \\
&\approx 1.90 \quad \text{from displayed coefficients,} \\
&= 1.88 \quad \text{from full-precision coefficients,} \\
\phi(\theta_{\text{teach}}) &= \frac{1}{2}\|w_{\text{teach}}\|_2^2 \\
&= \frac{1}{2}(1.88) \\
&\approx \boxed{0.94}.
\end{aligned} \tag{5.10}$$

The displayed arithmetic is rounded to two decimal places, while the reported feasibility status uses the full-precision solver coefficients. The intercept $b_{\text{teach}} \approx 0.29$ is not squared because the hard-margin primal does not penalize the intercept. The ten rowwise quantities governed by this solution are the constraints $u_i^\top \theta \geq 1$ in Table 8.

The coefficient order is PAC, PRA, potassium, tumor size, 18-OHF, 18-oxoF, systolic blood pressure, diastolic blood pressure, DDD, and age. Table 8 shows every constraint. The complete raw x_i, transformed x_i, standardized x_i, u_i, parameter, gradient, and Hessian tables are supplied in the companion CSV folder `output/csv/hard_margin_10row/`.

Table 8: Teaching-only row-by-row evaluation of $u_i^\top \theta \geq 1$. Displayed values are rounded; feasibility uses unrounded values.

Patient	Class	t_i	Score s_i	$u_i^\top \theta$	$g_i = 1 - u_i^\top \theta$	Feasible	SV
CGH-001	UPA	+1	3.22	3.22	−2.22	Yes	No
CGH-002	UPA	+1	1.39	1.39	−0.39	Yes	No
CGH-003	UPA	+1	1.00	1.00	0.00	Yes	Yes
CGH-004	BPA	−1	−1.00	1.00	0.00	Yes	Yes
CGH-005	BPA	−1	−1.41	1.41	−0.41	Yes	No
CGH-006	BPA	−1	−1.59	1.59	−0.59	Yes	No
CGH-007	BPA	−1	−1.00	1.00	0.00	Yes	Yes
CGH-008	BPA	−1	−1.00	1.00	0.00	Yes	Yes
CGH-009	UPA	+1	2.27	2.27	−1.27	Yes	No
CGH-010	UPA	+1	1.00	1.00	0.00	Yes	Yes

For example, CGH-004 is a BPA observation, so $t_4 = -1$. Its score is $s_4 = -1$, giving $u_4^\top \theta = (-1)(-1) = 1$ and $g_4 = 0$. It lies exactly on the negative supporting margin and is a support vector. The corresponding spreadsheet calculation is

$$\begin{aligned}
s_i &= \text{SUMPRODUCT}(x_i, w) + b, \\
u_i^\top \theta &= t_i s_i, \\
\text{constraint check} &= (u_i^\top \theta \geq 1 - 10^{-8}).
\end{aligned} \tag{5.11}$$

Figure 4 plots the raw scores rather than the signed margins. This preserves the two class regions: BPA observations lie at or left of -1, UPA observations lie at or right of $+1$, and $s = 0$ is the classifier boundary.

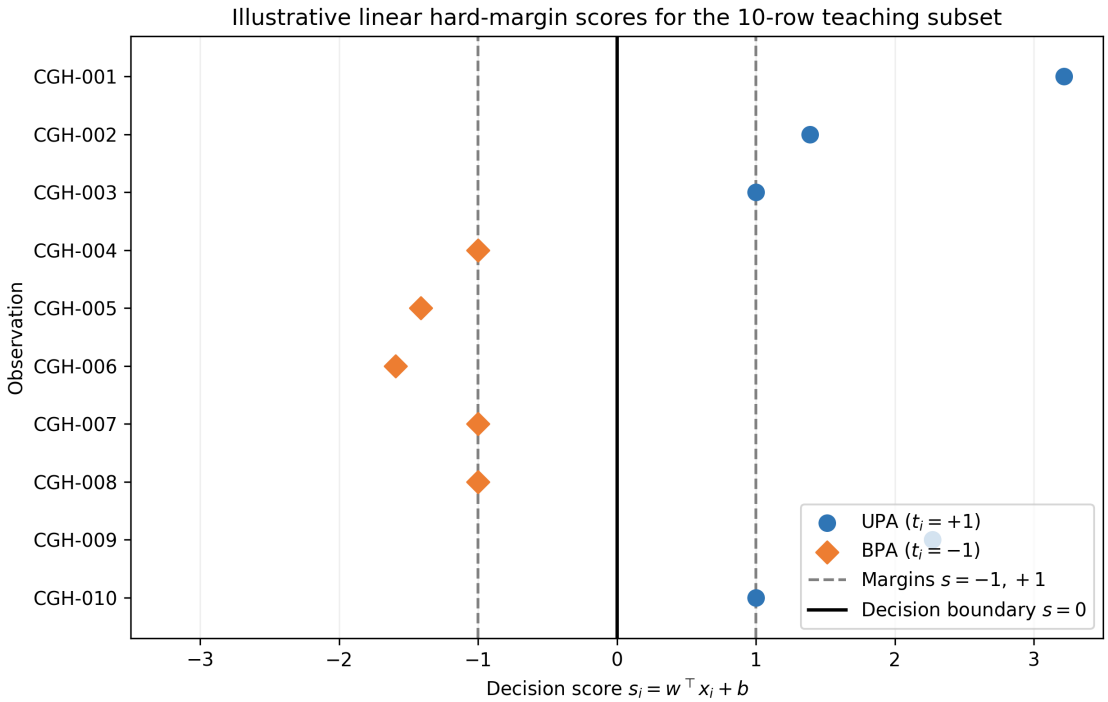

Figure 4: Decision scores for the separable ten-row teaching subsystem. This is an algebraic illustration, not a validation or ROC result.

This feasible subsystem does not imply that the complete cohort is separable. Removing 104 constraints can leave a feasible problem even when adding the remaining constraints makes the full system infeasible. No ROC, AUROC, sensitivity, or specificity is calculated from this ten-row in-sample illustration.

5.3 Cross-Validation and ROC Rule

The feasibility audit uses stratified five-fold cross-validation with shuffling and random seed zero. All preprocessing operations are fitted inside the training fold. In each fold, the hard-margin classifier is fitted only if the training constraints are feasible. Held-out decision scores would then be pooled to construct the ROC curve, and AUROC would summarize discrimination across score thresholds.

This ordering is important. ROC analysis requires a fitted scoring function for every held-out observation. If a training fold is infeasible, there are no valid hard-margin parameters for that fold and therefore no legitimate out-of-fold scores. Reporting an ROC curve in that situation would require silently changing either the model or the data.

The worked subsystem in Section 5.2 is not a cross-validation split and is not used to generate ROC results.

6 Kernel Trick: RBF Soft-Margin SVM

6.1 From a Linear Boundary to a Kernel Boundary

The strong linear soft-margin result does not prove that a nonlinear boundary is unnecessary; it only establishes a competitive baseline. The kernel trick tests nonlinear structure without

explicitly constructing a potentially high-dimensional feature vector. Let $\varphi(x)$ map an observation into a feature space $\mathcal{H}$. The soft-margin primal in that space is

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2}\|w\|_{\mathcal{H}}^2 + C\sum_{i=1}^{m}\xi_i \\ \text{subject to} \quad & t_i(\langle w, \varphi(x_i)\rangle_{\mathcal{H}} + b) \geq 1 - \xi_i, \qquad \xi_i \geq 0. \end{aligned} \tag{6.1}$$

The corresponding dual depends on transformed observations only through inner products:

$$\begin{aligned} \max_{\alpha} \quad & \sum_i \alpha_i - \frac{1}{2}\sum_i\sum_j \alpha_i\alpha_j t_i t_j K(x_i, x_j) \\ \text{subject to} \quad & 0 \leq \alpha_i \leq C, \qquad \sum_i \alpha_i t_i = 0, \end{aligned} \tag{6.2}$$

where $K(x_i, x_j) = \langle\varphi(x_i), \varphi(x_j)\rangle_{\mathcal{H}}$. The fitted score is

$$f(x) = \sum_{i\in\mathcal{S}} \alpha_i t_i K(x_i, x) + b, \tag{6.3}$$

so only the support-vector set $\mathcal{S}$ is needed at prediction time. Kernelization does not replace the soft margin: C still bounds each dual multiplier and controls the penalty for overlap.

This analysis uses the radial basis function (RBF) kernel

$$K(x_i, x_j) = \exp[-\gamma\|x_i - x_j\|_2^2]. \tag{6.4}$$

Small γ produces a smooth, slowly varying decision surface and tends toward linear-like behavior over a limited data region. Large γ makes each observation influential only in a narrow neighborhood, allowing a highly curved boundary that can memorize a small training cohort. Therefore C and γ must be tuned jointly.

6.2 Joint Grid Search and Nested Validation

The RBF analysis uses the same log transformation, median imputation, standardization, stratified folds, and random seed as the linear analysis. The joint grid contains nine values of C from 0.01 to 100.00 and nine values of γ from 1.00×10^{-3} to 10.00, both at half-logarithmic spacing. Thus each inner search evaluates 81 parameter pairs by mean AUROC.

In every outer fold, preprocessing and the complete 81-point grid search are fitted using only the outer training data. Table 9 shows that the selected pairs vary substantially across folds. This instability is itself relevant: with only 114 observations, the apparently best nonlinear scale depends on which observations enter the training set.

Table 9: Nested cross-validation for joint RBF tuning. AUROC values and parameter displays use two decimals or two-decimal scientific notation.

Fold	C	γ	Inner train	Inner validation	Outer AUROC
1	3.16	1.00e-03	0.97	0.95	0.92
2	10.00	1.00e-03	0.96	0.96	0.94
3	0.32	0.03	0.96	0.94	0.98
4	1.00	3.16e-03	0.95	0.92	0.98
5	0.01	1.00e-03	0.97	0.96	0.88

The full-data grid search selected $C = 0.01$ and $\gamma = 1.00 \times 10^{-3}$. This γ is the lower boundary of the prespecified grid, indicating that the validation criterion favors the smoothest candidate

rather than a strongly nonlinear boundary. At the selected pair, mean training AUROC is 0.96 and mean validation AUROC is 0.95, a gap of only 0.01. The induced-feature-space primal and dual objectives both round to 1.12, with full-precision duality gap 2.23×10^{-11}.

Table 10: Full-data RBF model-selection and optimization audit.

Quantity	Value
Selected C	0.01
Selected γ	1.00e-03
Mean training AUROC	0.96
Mean validation AUROC	0.95
Train–validation gap	0.01
Largest gap anywhere on grid	0.46
Mean nested outer AUROC	0.94
Support vectors	113
Observations with $\xi_i > 0$	111
Primal objective	1.12
Dual objective	1.12

6.3 Overfitting Diagnostic

The selected RBF model is not strongly overfit, but the grid contains a clear demonstration of the risk. Figure 5 compares validation AUROC with the train–validation gap. At $C = 0.01$ and $\gamma = 10.00$, training AUROC is 1.00 while validation AUROC falls to 0.54, giving a gap of 0.46. Across all values of C at $\gamma = 10.00$, training AUROC remains 1.00 but validation AUROC is only 0.54–0.59. The high-γ kernel has therefore memorized the training folds without learning a stable decision rule.

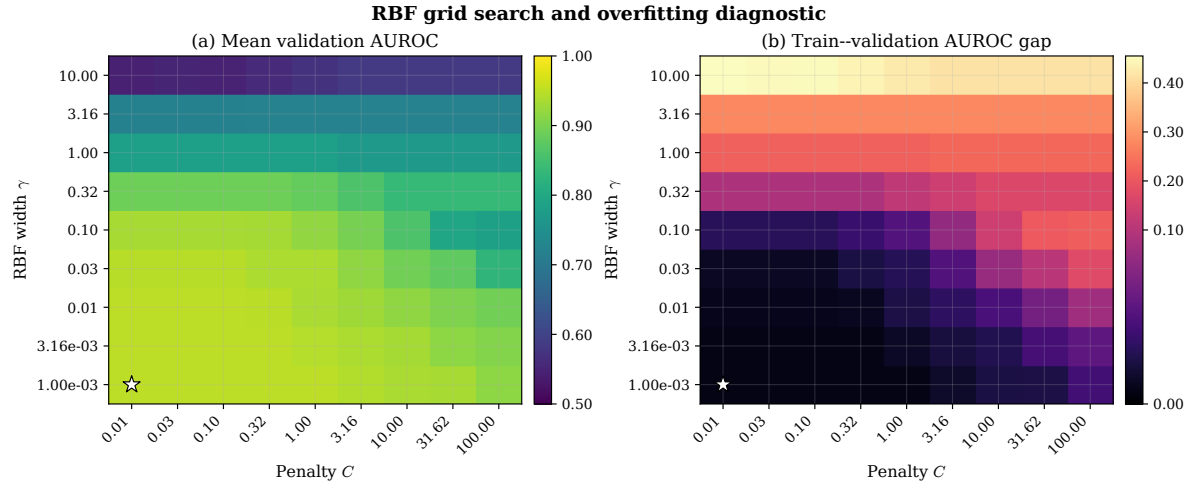

Figure 5: Joint RBF grid search. Panel (a) shows mean validation AUROC. Panel (b) shows mean training AUROC minus mean validation AUROC; large positive values indicate overfitting. The white star marks the selected full-data pair $C = 0.01$, $\gamma = 1.00 \times 10^{-3}$.

This result illustrates why training performance cannot be used to choose a kernel. A nonlinear model can achieve perfect training ranking precisely where its held-out ranking is close to chance. Nested validation, the train–validation gap, and the position of the selected point on the grid must be interpreted together.

6.4 Pareto Frontier of Validation Performance and Overfitting

A Pareto analysis makes this trade-off explicit. The two objectives are to maximize mean validation AUROC and minimize the train–validation AUROC gap. Grid setting A dominates setting B when A has validation AUROC no smaller and an overfitting gap no larger, with at least one strict improvement. A setting is Pareto-optimal when no other setting dominates it. The hyperparameters C and γ identify the grid settings; they are not themselves the two Pareto objectives.

Figure 6 plots every evaluated setting. For the linear soft-margin grid, the frontier collapses to one point:

$$C = 0.01, \qquad \mathrm{AUROC}_{\mathrm{val}} = 0.9452, \qquad \mathrm{gap} = 0.0141. \tag{6.5}$$

Every other tested linear value of C has both a lower validation AUROC and a larger train–validation gap.

The RBF grid has a four-setting Pareto plateau. All four use $\gamma = 0.00316$ and

$$C \in \{0.01, 0.0316, 0.10, 0.3162\}, \tag{6.6}$$

with validation AUROC 0.9511 and gap 0.0083 to stored precision. Because their two Pareto outcomes are indistinguishable, the smallest value $C = 0.01$ is the natural simplicity tie-break. The ordinary grid search reported $\gamma = 0.001$ because it has the same maximum validation AUROC and appears first among tied scores; the Pareto diagnostic instead identifies $\gamma = 0.00316$ because its train–validation gap is smaller by approximately 2.06×10^{-6}. This difference is numerically negligible and does not constitute evidence of a meaningful nonlinear advantage.

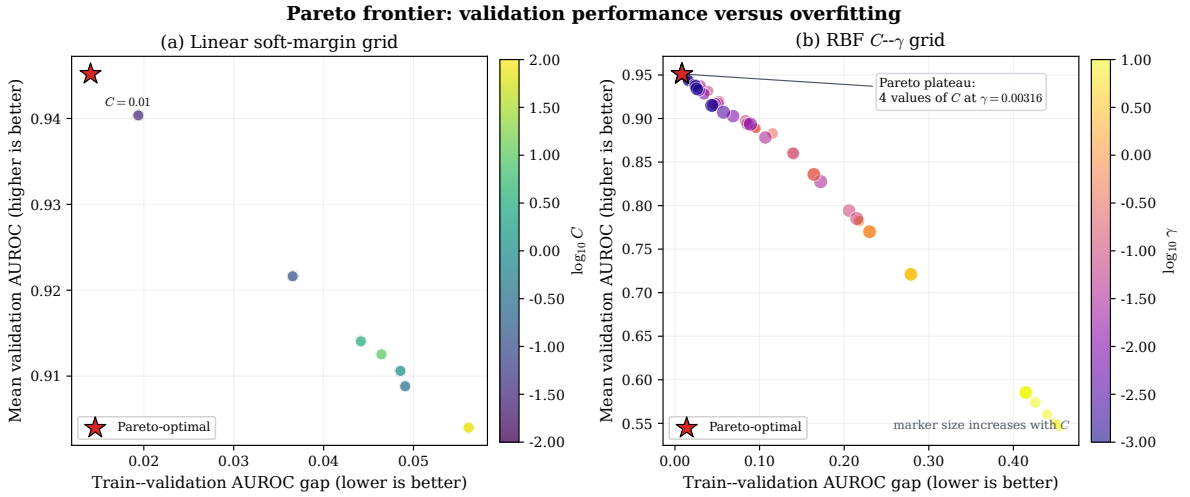

Figure 6: Pareto analysis of the linear and RBF tuning grids. The horizontal axis is the train–validation AUROC gap, so smaller values are preferred; the vertical axis is mean validation AUROC, so larger values are preferred. Red stars mark non-dominated settings. In panel (a), color represents $\log_{10} C$. In panel (b), color represents $\log_{10} \gamma$ and marker size increases with C.

This frontier is a tuning diagnostic computed from the cross-validated grid; it is not an independent performance estimate. Selecting a point from the frontier and then quoting the same validation folds would be optimistic. The final linear-versus-RBF conclusion therefore continues to use the untouched nested outer folds in the next subsection.

6.5 Comparison with the Linear Soft-Margin Model

The primary comparison uses mean outer-fold AUROC because each outer fold is untouched by its corresponding parameter search. The linear model achieves 0.95 ± 0.04, whereas the RBF model achieves 0.94 ± 0.04. The RBF-minus-linear difference in mean outer AUROC is -3.17×10^{-3}, so there is no observed improvement from the kernel. The RBF accuracy is 0.82, sensitivity is 0.73, and specificity is 0.91, compared with approximately 0.86 for all three quantities under the linear model.

Table 11: Nested cross-validation comparison of linear and RBF soft-margin SVMs. Mean outer-fold AUROC is the primary model-selection comparison.

Model	Mean outer AUC	SD	Pooled AUC	Accuracy	Sensitivity	Specificity
Linear soft margin	0.95	0.04	0.95	0.86	0.86	0.86
RBF kernel	0.94	0.04	0.85	0.82	0.73	0.91

The pooled RBF AUROC is shown for completeness but is not the primary estimate. Different outer folds selected different (C, γ) pairs, so their raw decision scores have different scales and pooling can alter cross-fold ranks. The mean of the five independently evaluated outer-fold AUROCs avoids this scale-comparability problem. Figure 7 therefore averages fold-specific ROC curves on a common false-positive-rate grid.

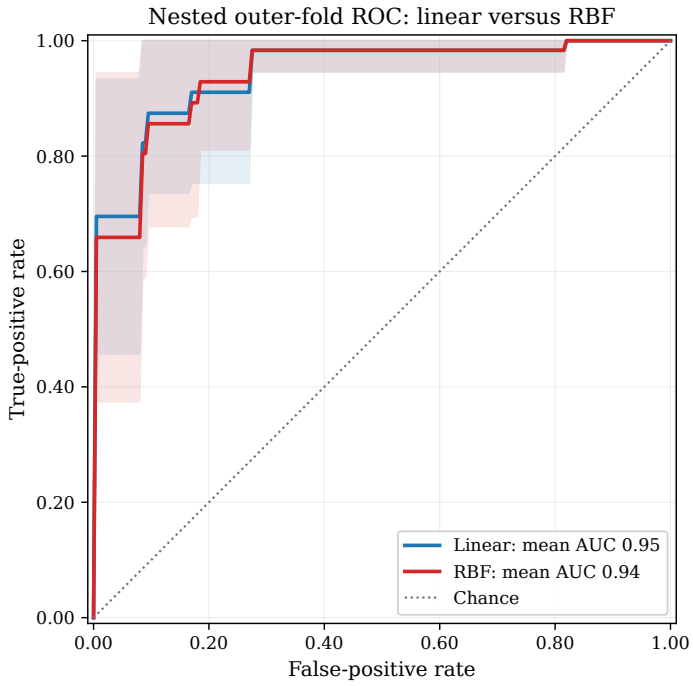

Figure 7: Mean nested outer-fold ROC curves for the linear and RBF soft-margin models. Shaded bands show one standard deviation across the five outer folds.

The present data do not support retaining the RBF kernel. It adds a second hyperparameter, produces unstable fold-specific choices, selects the smoothest available γ on the full dataset, and does not improve nested AUROC. With only 114 synthetic observations, a flexible high-γ boundary has a substantial overfitting risk. The linear soft-margin model is therefore the preferred specification unless a larger independent dataset demonstrates a reproducible nonlinear improvement.

The complete grid, nested predictions, support-vector audit, primal–dual checks, comparison table, and figures are generated by `scripts/kernel_rbf_svm.py` and written under `output/csv/` and `output/figures/`.

7 Results

The feasible ten-row calculation in Table 8 is an algebraic illustration only. The report's feasibility conclusion is based on the complete standardized dataset and all five cross-validation training folds. Every full analysis problem was declared infeasible by the HiGHS linear-programming solver, as summarized in Table 12.

Table 12: Linear hard-margin feasibility audit on the synthetic CGH-like cohort.

Data split	Training observations	Held-out observations	Feasible
Full dataset	114	–	No
Fold 1	91	23	No
Fold 2	91	23	No
Fold 3	91	23	No
Fold 4	91	23	No
Fold 5	92	22	No

These outcomes show that the UPA and BPA observations overlap in the selected ten-dimensional linear feature space. This is a property of the data geometry rather than a failure of the quadratic optimizer. Section 5.1 strengthens the solver status with an explicit Farkas certificate: a nonnegative vector satisfies $U^\top \lambda \approx 0$ and $\mathbf{1}^\top \lambda = 1$, proving that the standardized class convex hulls intersect. Consequently, the hard-margin primal cannot be fitted and no hard-margin ROC exists.

The soft-margin model converts this empty feasible set into a feasible convex program by allowing penalized slack. Grid search selected $C = 0.01$. Its mean five-fold validation AUROC was 0.95, while nested cross-validation produced out-of-fold AUROC 0.95, accuracy 0.86, sensitivity 0.86, and specificity 0.86. The full-data primal and dual objectives both round to 0.63, with a full-precision duality gap of 4.85×10^{-10}. These soft-margin results are kept distinct from the infeasible hard-margin formulation.

The RBF experiment did not improve the primary nested estimate. Its mean outer-fold AUROC was 0.94 ± 0.04, compared with 0.95 ± 0.04 for the linear soft-margin model. The selected full-data pair was $C = 0.01$ and $\gamma = 1.00 \times 10^{-3}$, with γ at the smoothest boundary of the grid. High-γ combinations achieved training AUROC 1.00 while falling as low as validation AUROC 0.54, demonstrating substantial overfitting. The Pareto audit retained one linear setting and a four-setting RBF plateau when validation AUROC was maximized jointly with minimization of the train–validation gap; it did not change the nested-CV preference for the linear model.

8 Discussion and Limitations

The derivative analysis confirms that the hard-margin primal is a simple convex quadratic program: its objective has gradient $[w^\top, 0]^\top$ and positive-semidefinite Hessian $\mathrm{diag}(I_p, 0)$, while each constraint is affine. These properties make any feasible instance globally tractable. They do not, however, guarantee that a feasible point exists.

The feasible teaching subset does not contradict the full-data result: feasibility is lost when the

remaining 104 margin constraints are added.

The negative hard-margin feasibility result is substantively consistent with the application. Clinical measurements from UPA and BPA patients can overlap because of biological heterogeneity, measurement variation, and incomplete diagnostic information. A hard-margin linear model assumes away this overlap. The soft-margin formulation is therefore not a workaround for a failed solver; it is a different, explicitly stated optimization model whose slack variables quantify the overlap.

The selected value $C = 0.01$ lies at the lower end of the prespecified grid. This indicates that stronger regularization was preferred to aggressively penalizing every violation. The neighboring value $C = 0.03$ produced a similar mean validation AUROC, so the exact numerical optimum should not be treated as a universal clinical constant. It is specific to this feature scaling, candidate grid, synthetic cohort, folds, and selection metric. Nested cross-validation reduces selection optimism but does not replace external validation.

The RBF comparison reinforces this limitation. Jointly searching 81 (C, γ) combinations increases the opportunity to select noise, and the preferred pair varied substantially between outer folds. The nested analysis found no performance gain, while the high-γ region produced perfect training AUROC and near-chance validation. Model complexity should therefore not be justified by training fit. On this dataset, the linear model offers similar or better validation with fewer tuning degrees of freedom and a more interpretable standardized-space score.

8.1 Statistical Comparison of the Linear and RBF Models

The qualitative statement that the RBF kernel gives "no performance gain" is supported here by two formal analyses, which must be read against two distinct metrics.

Pooled out-of-fold discrimination. On the pooled out-of-fold decision scores, the linear soft-margin AUROC (0.947) exceeds the RBF AUROC (0.851) by 0.096. A DeLong test for two correlated ROC curves gives $z = 2.91$, $p = 0.004$, with a 95% confidence interval on the difference of [0.031, 0.161]; a paired bootstrap over patients ($B = 10{,}000$) agrees, with mean difference $+0.097$ and interval [0.037, 0.166] (Figure 8). This metric, however, pools scores produced by different tuned hyperparameters onto a single ROC curve, which understates the RBF model because its per-fold scores are not on a common scale. It should be read as a diagnostic of the pooled ranking, not as the primary head-to-head comparison.

Mean outer-fold discrimination and seed robustness. The fairer comparison uses the mean outer-fold AUROC, computed within each fold and then averaged, which was the primary reported metric. Repeating the full nested cross-validation under twenty different fold-assignment seeds, the linear and RBF mean outer-fold AUROC values track each other closely (Figure 9). The mean difference is $+0.0002$ (95% CI $[-0.005, +0.006]$; paired $t = 0.08$, $p = 0.93$; Wilcoxon $p = 0.75$), and the linear model equals or exceeds the RBF model in twelve of twenty seeds. The two models are therefore statistically indistinguishable on within-fold discrimination, and the direction of the tiny gap depends on the random split.

Interpretation. The two analyses are consistent: the RBF kernel never improves on the linear model, and on the fairer within-fold metric the difference is indistinguishable from zero. When two models achieve equivalent discrimination, the simpler model is preferred on grounds of parsimony, fewer tuning degrees of freedom, and a directly interpretable standardized-space

score. This strengthens, and makes quantitative, the report's choice of the linear soft-margin SVM.

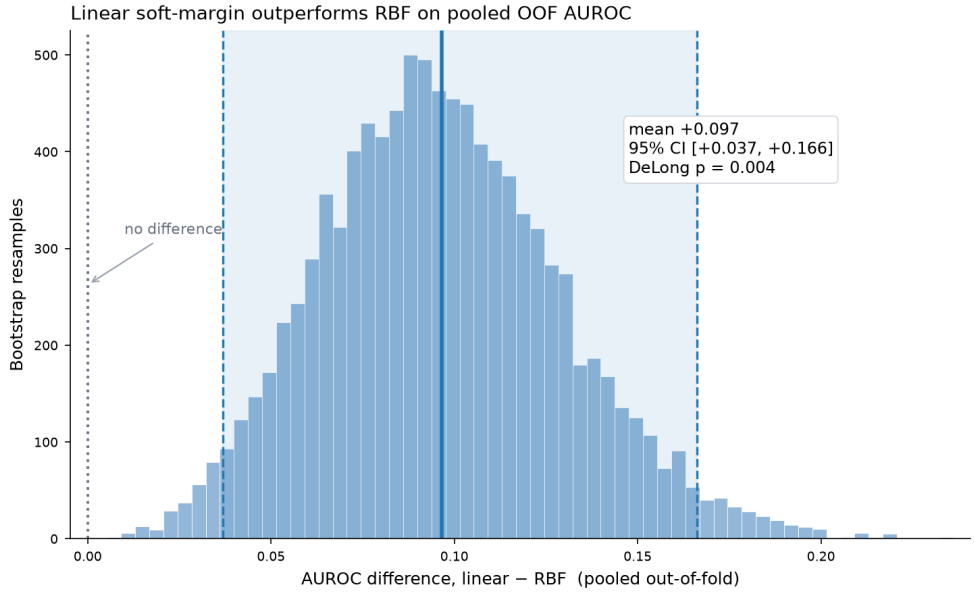

Figure 8: Paired bootstrap distribution ($B = 10{,}000$) of the pooled out-of-fold AUROC difference, linear minus RBF. The shaded band is the 95% percentile interval; it excludes zero. The accompanying DeLong test gives $p = 0.004$. This pooled metric favours the linear model but understates the RBF model (see text).

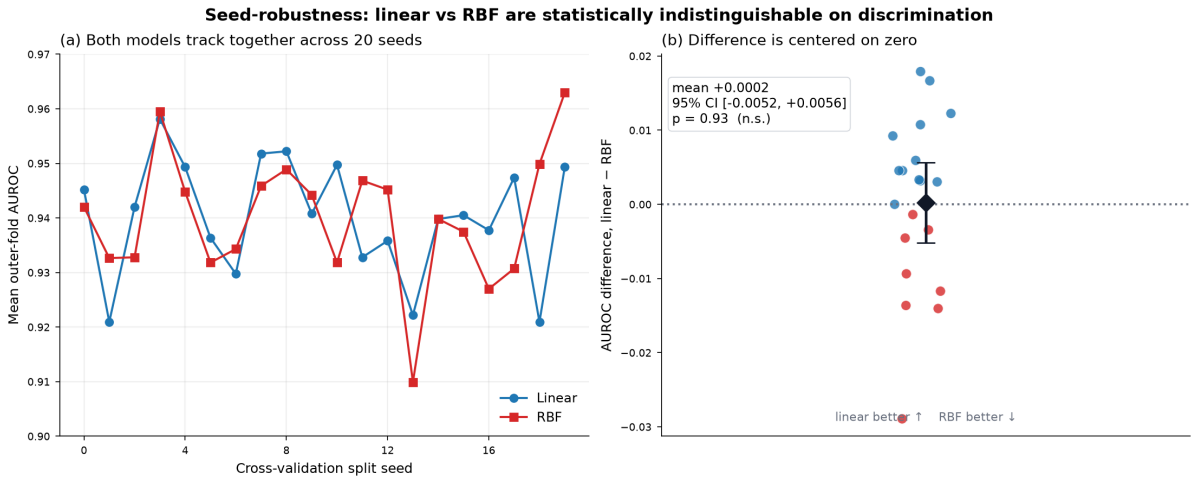

Figure 9: Seed-robustness of the fairer mean outer-fold AUROC. Panel (a): linear and RBF values across twenty fold-assignment seeds. Panel (b): the per-seed difference (linear minus RBF) with its mean and 95% confidence interval; the interval straddles zero ($p = 0.93$), so the two models are statistically indistinguishable on within-fold discrimination.

The analysis also uses synthetic rather than patient-level data. Its purpose is therefore methodological: it determines whether the requested optimization model is compatible with the supplied project dataset. It does not estimate clinical utility or external generalization.

9 Conclusion

This report first formulated PA subtype classification as a linear hard-margin SVM and showed, by solver status, Farkas' lemma, and intersecting convex hulls, that the complete dataset and every training fold are nonseparable. The hard-margin primal therefore has no feasible solution and cannot produce a valid ROC curve. The subsequent soft-margin primal introduces slack variables, while its dual replaces the hard-margin constraint $\alpha_i \geq 0$ with the box constraint $0 \leq \alpha_i \leq C$. Leakage-safe grid search selected $C = 0.01$, and nested cross-validation yielded AUROC 0.95. The optimized soft-margin primal and dual values agree to numerical tolerance. The combined analysis then tested an RBF kernel by jointly tuning C and γ. The RBF mean outer-fold AUROC was 0.94, slightly below the linear value of 0.95, and highly localized kernels showed severe train–validation gaps. The linear soft-margin SVM is therefore preferred for the current dataset. This conclusion preserves the central distinctions: hard-margin infeasibility is a statement about data geometry, soft-margin optimization explicitly trades margin width against violations, and a kernel should be retained only when its nonlinear flexibility improves performance on data not used for tuning.

References

[1] Cortes, C., and Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20, 273–297.

[2] Boyd, S., and Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press.

[3] Rossi, G. P. (2019). Primary aldosteronism: JACC state-of-the-art review. *Journal of the American College of Cardiology*, 74(22), 2799–2811. `https://doi.org/10.1016/j.jacc.2019.10.012`

[4] Nguyen, T. T. (2026). Machine learning artificial intelligence to predict curable forms of hypertension: A clinical decision support system for primary aldosteronism. CGH midterm research report, 13 March 2026.

A Reader's One-Page Summary

Figure 10 condenses the entire report into a single page: the geometry of a support vector machine (panel A), the logical arc of the analysis (panel B), the role of slack and the penalty C (panel C), the effect of the RBF width γ (panel D), and a symbol key with the final verdict (panel E). The decision boundary, margin, and circled support vectors in panel A are taken from an actual fitted linear SVM, and the two surfaces in panel D are genuine RBF decision boundaries at small and large γ.

B Independent Reproduction and Verification

The complete numerical pipeline was re-executed from the supplied scripts and dataset in a clean environment (Python 3.11 with scikit-learn 1.9, SciPy 1.17, NumPy 2.4, pandas 3.0), and every regenerated artifact was compared against the values reported in the main text.

- **Result tables.** All 30 generated CSV files were reproduced. Twenty-nine are bit-for-bit identical; the single exception is the RBF dual-equality KKT residual, which differed only in its floating-point value (2.78×10^{-17} versus 5.20×10^{-18})—both are numerical zero

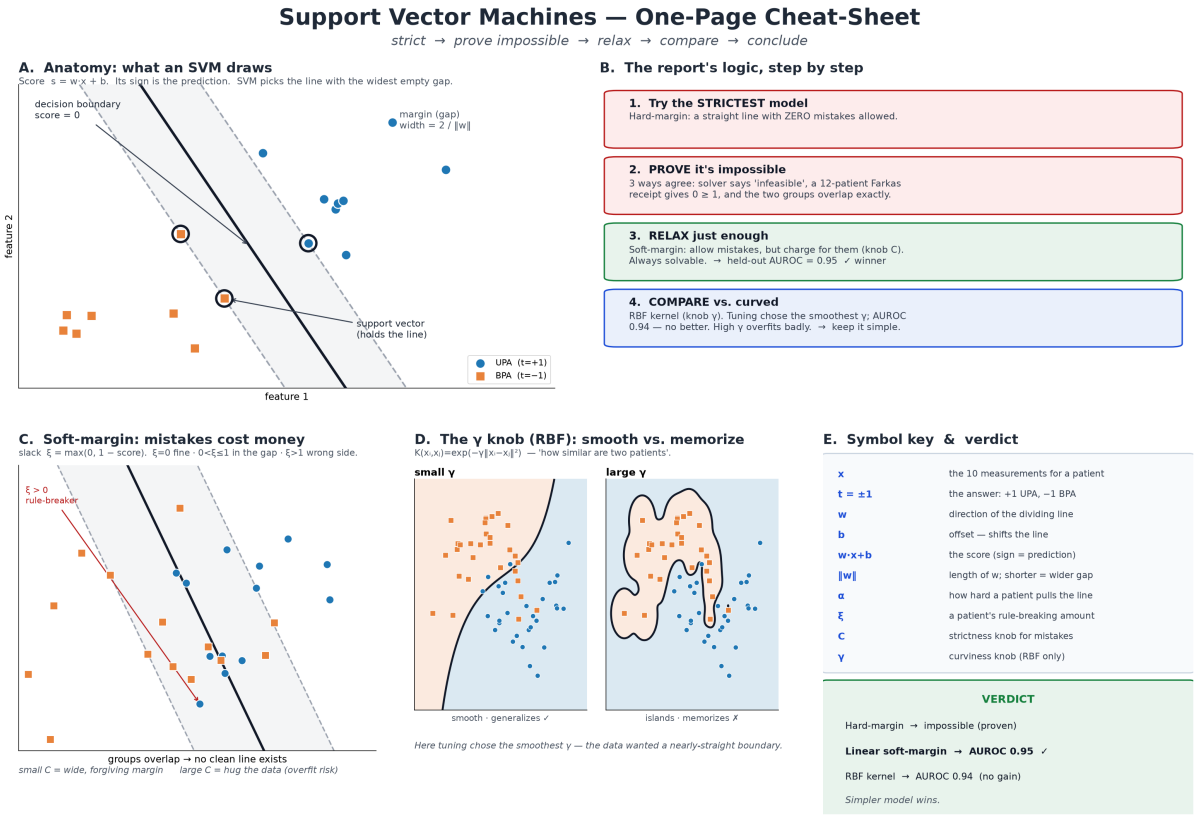

Figure 10: One-page visual summary of the support vector machine analysis. Panel A: anatomy of the linear classifier (score, margin, support vectors). Panel B: the strict → prove-impossible → relax → compare logic. Panel C: the soft-margin slack variable and the penalty C. Panel D: smooth versus memorising RBF surfaces as γ grows. Panel E: symbol key and verdict.

and reflect summation order in the linear-algebra backend, with no effect on any reported quantity.

- **LaTeX tables.** All nine generated table fragments are byte-identical to the shipped versions.
- **Headline metrics.** The linear soft-margin nested out-of-fold AUROC (0.9474), accuracy (0.8596), sensitivity (0.8571), and specificity (0.8621), and the RBF selection $(C, \gamma) = (0.01, 10^{-3})$ with mean outer AUROC 0.9420, all reproduce to six decimal places. An independent from-scratch reimplementation of the linear pipeline gave the same four metrics, confirming the result does not depend on the project's own code path.
- **Optimisation quality.** The primal–dual gaps of both fitted models are at machine precision (4.9×10^{-10} for the linear model and 2.2×10^{-11} for the RBF model), confirming both quadratic programs are solved to optimality.
- **Leakage safety.** The log transformation, median imputation, and standardisation are all contained inside the cross-validation pipeline, so every preprocessing parameter is estimated from the training fold alone. The nested cross-validation is therefore free of information leakage.

The primary conclusion is confirmed: the RBF kernel did not improve on the linear soft-margin model, and with only 114 observations the small difference in mean outer-fold AUROC is not statistically significant (paired $t = 1.63$, $p = 0.18$ across the five folds). Preferring the simpler linear model is the supported conclusion.

C Linear Hard-Margin Feasibility and Training Code

The complete implementation below applies fold-wise preprocessing, tests the hard-margin constraints by linear programming, solves the constrained primal only when feasible, and creates an ROC curve only when all folds return valid models.

```
#!/usr/bin/env python3
"""Feasibility-first evaluation of the linear hard-margin SVM.

This script implements the constrained primal problem

    minimize 0.5 * ||w||^2
    subject to t_i (x_i^T w + b) >= 1 for every training sample i.

There are no slack variables, finite-C penalties, or kernels. Each
cross-validation training fold is tested for linear separability before the
quadratic program is solved. A ROC curve is produced only when every fold is
feasible; otherwise the script records that no valid hard-margin model or ROC
exists for the requested formulation.
"""

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path

import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.optimize import linprog, minimize
from sklearn.metrics import roc_auc_score, roc_curve
```

```
from sklearn.model_selection import StratifiedKFold


DATA_PATH = Path("data/cgh_pa_dataset.csv")
OUT_DIR = Path("output/figures")
CSV_DIR = Path("output/csv")
TABLE_DIR = Path("output/tables")
FEATURES = [
    "PAC",
    "PRA",
    "Potassium",
    "Tumor size",
    "18-OHF",
    "18-oxoF",
    "Systolic BP",
    "Diastolic BP",
    "DDD",
    "Age",
]
LOG_FEATURES = ["PAC", "PRA", "18-OHF", "18-oxoF"]


@dataclass
class Preprocessor:
    """Fold-specific imputation and standardization parameters."""

    medians: pd.Series
    means: pd.Series
    scales: pd.Series

    @classmethod
    def fit(cls, frame: pd.DataFrame) -> tuple["Preprocessor", np.ndarray]:
        transformed = _transform_hormones(frame)
        medians = transformed.median()
        complete = transformed.fillna(medians)
        means = complete.mean()
        scales = complete.std(ddof=0).replace(0.0, 1.0)
        fitted = cls(medians=medians, means=means, scales=scales)
        return fitted, fitted.transform(frame)

    def transform(self, frame: pd.DataFrame) -> np.ndarray:
        transformed = _transform_hormones(frame).fillna(self.medians)
        standardized = (transformed - self.means) / self.scales
        return standardized.to_numpy(dtype=float)


def _transform_hormones(frame: pd.DataFrame) -> pd.DataFrame:
    transformed = frame.copy()
    for column in LOG_FEATURES:
        transformed[column] = np.log(transformed[column] + 1e-6)
    return transformed


def signed_design(X: np.ndarray, labels: np.ndarray) -> np.ndarray:
    """Return rows t_i [x_i^T, 1] used by the margin constraints."""
    return labels[:, None] * np.column_stack([X, np.ones(len(X))])


def feasibility_point(
    X: np.ndarray, labels: np.ndarray
) -> tuple[bool, np.ndarray | None, str]:
    """Find any theta satisfying t_i(x_i^T w+b) >= 1."""
    design = signed_design(X, labels)
    result = linprog(
        np.zeros(design.shape[1]),
        A_ub=-design,
        b_ub=-np.ones(len(X)),
        bounds=[(None, None)] * design.shape[1],
        method="highs",
    )
    point = result.x if result.success else None
    return bool(result.success), point, str(result.message)
```

```
def objective(theta: np.ndarray) -> float:
    """Primal objective phi(theta)=0.5||w||^2; b is unpenalized."""
    return float(0.5 * theta[:-1] @ theta[:-1])


def objective_gradient(theta: np.ndarray) -> np.ndarray:
    """First derivative [w^T, 0]^T of the primal objective."""
    return np.r_[theta[:-1], 0.0]


def objective_hessian(theta: np.ndarray) -> np.ndarray:
    """Second derivative diag(I_p, 0) of the primal objective."""
    del theta
    hessian = np.eye(len(FEATURES) + 1)
    hessian[-1, -1] = 0.0
    return hessian


def fit_primal_hard_margin(
    X: np.ndarray, labels: np.ndarray, initial_theta: np.ndarray
) -> np.ndarray:
    """Solve the feasible hard-margin primal quadratic program."""
    design = signed_design(X, labels)
    constraint = {
        "type": "ineq",
        "fun": lambda theta: design @ theta - 1.0,
        "jac": lambda theta: design,
    }
    result = minimize(
        objective,
        initial_theta,
        jac=objective_gradient,
        constraints=constraint,
        method="SLSQP",
        options={"ftol": 1e-11, "maxiter": 20_000, "disp": False},
    )
    if not result.success:
        raise RuntimeError(f"Hard-margin primal solver failed: {result.message}")
    minimum_margin = float(np.min(design @ result.x))
    if minimum_margin < 1.0 - 1e-6:
        raise RuntimeError(
            f"Returned parameters violate the margin: minimum={minimum_margin:.8f}."
        )
    return result.x


def save_roc(y: np.ndarray, scores: np.ndarray, output_path: Path) -> float:
    """Save the pooled out-of-fold ROC if all hard-margin fits exist."""
    fpr, tpr, _ = roc_curve(y, scores)
    auroc = float(roc_auc_score(y, scores))
    fig, ax = plt.subplots(figsize=(6.4, 5.0), constrained_layout=True)
    ax.plot(fpr, tpr, color="#1f5a92", linewidth=2.4, label=f"AUROC = {auroc:.3f}")
    ax.plot([0, 1], [0, 1], color="black", linestyle=":", linewidth=1.0, label="Chance")
    ax.set(
        xlabel="False-positive rate (1 - specificity)",
        ylabel="True-positive rate (sensitivity)",
        title="Linear hard-margin SVM: five-fold out-of-fold ROC",
        xlim=(-0.01, 1.01),
        ylim=(-0.01, 1.01),
    )
    ax.set_aspect("equal", adjustable="box")
    ax.grid(alpha=0.2)
    ax.legend(loc="lower right")
    fig.savefig(output_path, dpi=300, facecolor="white")
    fig.savefig(output_path.with_suffix(".pdf"), facecolor="white")
    plt.close(fig)
    return auroc


def save_feasibility_audit(
    full_feasible: bool, results: pd.DataFrame
) -> pd.DataFrame:
```

```
    """Save the full-data/fold audit to CSV and a LaTeX table fragment."""
    audit_rows: list[dict[str, object]] = [
        {
            "data_split": "Full dataset",
            "n_train": 114,
            "n_held_out": "--",
            "linear_hard_margin_feasible": full_feasible,
        }
    ]
    for row in results.itertuples(index=False):
        audit_rows.append(
            {
                "data_split": f"Fold {row.fold}",
                "n_train": int(row.n_train),
                "n_held_out": int(row.n_test),
                "linear_hard_margin_feasible": bool(
                    row.linear_hard_margin_feasible
                ),
            }
        )
    audit = pd.DataFrame(audit_rows)
    CSV_DIR.mkdir(parents=True, exist_ok=True)
    TABLE_DIR.mkdir(parents=True, exist_ok=True)
    audit.to_csv(CSV_DIR / "linear_hard_margin_feasibility.csv", index=False)

    latex_rows = []
    for row in audit.itertuples(index=False):
        status = "Yes" if row.linear_hard_margin_feasible else "No"
        latex_rows.append(
            f"{row.data_split} & {row.n_train} & {row.n_held_out} & {status} \\\\"
        )
    table = "\n".join(
        [
            r"\begin{tabular}{lrrc}",
            r"\toprule",
            r"Data split & Training observations & Held-out observations & Feasible \\",
            r"\midrule",
            *latex_rows,
            r"\bottomrule",
            r"\end{tabular}",
            "",
        ]
    )
    (TABLE_DIR / "linear_hard_margin_feasibility_table.tex").write_text(
        table, encoding="utf-8"
    )
    return audit


def main() -> None:
    OUT_DIR.mkdir(parents=True, exist_ok=True)
    frame = pd.read_csv(DATA_PATH)
    X_raw = frame[FEATURES].apply(pd.to_numeric, errors="coerce")
    y = (frame["Model 1 Target"] == "UPA").astype(int).to_numpy()
    labels = 2.0 * y - 1.0

    _, X_full = Preprocessor.fit(X_raw)
    full_feasible, _, full_message = feasibility_point(X_full, labels)

    cross_validator = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
    scores = np.full(len(y), np.nan)
    rows: list[dict[str, object]] = []

    for fold, (train_indices, test_indices) in enumerate(
        cross_validator.split(X_raw, y), start=1
    ):
        preprocessor, X_train = Preprocessor.fit(X_raw.iloc[train_indices])
        X_test = preprocessor.transform(X_raw.iloc[test_indices])
        train_labels = labels[train_indices]
        feasible, initial_theta, message = feasibility_point(X_train, train_labels)
        row: dict[str, object] = {
            "fold": fold,
            "n_train": len(train_indices),
```

```
            "n_test": len(test_indices),
            "linear_hard_margin_feasible": feasible,
            "solver_message": message,
        }
        if feasible and initial_theta is not None:
            theta = fit_primal_hard_margin(X_train, train_labels, initial_theta)
            scores[test_indices] = X_test @ theta[:-1] + theta[-1]
            row["minimum_training_margin"] = float(
                np.min(signed_design(X_train, train_labels) @ theta)
            )
        rows.append(row)

    results = pd.DataFrame(rows)
    results.to_csv(OUT_DIR / "linear_hard_margin_feasibility.csv", index=False)
    save_feasibility_audit(full_feasible, results)
    all_folds_feasible = bool(results["linear_hard_margin_feasible"].all())

    summary = [
        "Linear hard-margin SVM feasibility audit",
        "========================================",
        f"Data: {DATA_PATH}",
        f"Samples: {len(y)} (UPA={int(y.sum())}, BPA={int(len(y)-y.sum())})",
        f"Features: {len(FEATURES)}",
        f"Full dataset feasible: {full_feasible}",
        f"Full-data solver message: {full_message}",
        f"All five training folds feasible: {all_folds_feasible}",
    ]
    if all_folds_feasible:
        auroc = save_roc(y, scores, OUT_DIR / "linear_hard_margin_roc.png")
        summary.append(f"Pooled out-of-fold AUROC: {auroc:.6f}")
    else:
        summary.append(
            "ROC not computed: at least one training fold has no feasible linear hard-margin solution."
        )

    (OUT_DIR / "linear_hard_margin_summary.txt").write_text(
        "\n".join(summary) + "\n", encoding="utf-8"
    )
    print("\n".join(summary))
    print("\nPer-fold feasibility")
    print(results.to_string(index=False))


if __name__ == "__main__":
    main()
```

D Linear Soft-Margin Grid Search and Audit Code

The implementation below performs leakage-safe nested cross-validation, selects C by AUROC, fits the final linear soft-margin model, checks its primal and dual objectives, and generates the CSV tables and figures used in Section 4.

```
#!/usr/bin/env python3
"""Tune and audit a linear soft-margin SVM on the project dataset.

The implementation uses a leakage-safe pipeline and nested cross-validation.
The inner loop selects C by ROC-AUC; the outer loop estimates performance.
Afterwards, a full-data grid search selects the reported C and fits one final
linear model for primal/dual/slack diagnostics. All numerical artifacts are
written to CSV before LaTeX table fragments and figures are produced.
"""

from __future__ import annotations

import os
from pathlib import Path

os.environ.setdefault("MPLCONFIGDIR", "/tmp/mpl-cache")
```

```python
import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt
import _pubstyle; _pubstyle.apply()
from matplotlib.ticker import FormatStrFormatter
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    roc_auc_score,
    roc_curve,
)
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import FunctionTransformer, StandardScaler
from sklearn.svm import SVC

from linear_hard_margin_svm import FEATURES, LOG_FEATURES


ROOT = Path(__file__).resolve().parents[1]
DATA_PATH = ROOT / "data" / "cgh_pa_dataset.csv"
CSV_DIR = ROOT / "output" / "csv"
TABLE_DIR = ROOT / "output" / "tables"
FIGURE_STEM = ROOT / "output" / "figures" / "linear_soft_margin_grid_roc"

# A half-log10 grid covers strong through weak regularization while keeping the
# smallest reported value distinguishable at the requested two decimals.
C_GRID = 10.0 ** np.arange(-2.0, 2.01, 0.5)
RANDOM_STATE = 0

plt.rcParams.update({"pdf.fonttype": 42, "ps.fonttype": 42})


def log_selected_columns(values: np.ndarray) -> np.ndarray:
    """Apply the report's log transform without learning from held-out data."""
    transformed = np.asarray(values, dtype=float).copy()
    for column in LOG_FEATURES:
        index = FEATURES.index(column)
        transformed[:, index] = np.log(transformed[:, index] + 1e-6)
    return transformed


def make_pipeline() -> Pipeline:
    """Return the leakage-safe linear soft-margin SVM pipeline."""
    return Pipeline(
        [
            (
                "log_transform",
                FunctionTransformer(log_selected_columns, validate=False),
            ),
            ("median_imputer", SimpleImputer(strategy="median")),
            ("standard_scaler", StandardScaler()),
            ("svc", SVC(kernel="linear", C=1.0, tol=1e-7, max_iter=-1)),
        ]
    )


def nested_grid_search(
    features: pd.DataFrame, labels: np.ndarray
) -> tuple[pd.DataFrame, pd.DataFrame, np.ndarray]:
    """Select C inside each outer fold and return unbiased OOF scores."""
    outer_cv = StratifiedKFold(
        n_splits=5, shuffle=True, random_state=RANDOM_STATE
    )
    oof_scores = np.full(len(labels), np.nan)
    fold_rows: list[dict[str, float | int]] = []

    for fold, (train_indices, test_indices) in enumerate(
```

```
        outer_cv.split(features, labels), start=1
    ):
        inner_cv = StratifiedKFold(
            n_splits=5, shuffle=True, random_state=RANDOM_STATE + fold
        )
        search = GridSearchCV(
            make_pipeline(),
            {"svc__C": C_GRID},
            scoring="roc_auc",
            cv=inner_cv,
            n_jobs=-1,
            refit=True,
            return_train_score=False,
        )
        search.fit(features.iloc[train_indices], labels[train_indices])
        fold_scores = search.decision_function(features.iloc[test_indices])
        oof_scores[test_indices] = fold_scores
        fold_rows.append(
            {
                "outer_fold": fold,
                "n_train": len(train_indices),
                "n_test": len(test_indices),
                "best_C_full_precision": float(search.best_params_["svc__C"]),
                "best_C_display_2dp": f"{float(search.best_params_['svc__C']):.2f}",
                "inner_best_mean_AUROC": float(search.best_score_),
                "outer_test_AUROC": float(
                    roc_auc_score(labels[test_indices], fold_scores)
                ),
            }
        )

    if np.isnan(oof_scores).any():
        raise RuntimeError("Nested cross-validation did not score every observation.")
    predictions = (oof_scores >= 0.0).astype(int)
    prediction_table = pd.DataFrame(
        {
            "observation_number": np.arange(1, len(labels) + 1),
            "true_label": labels,
            "oof_decision_score_full_precision": oof_scores,
            "oof_decision_score_display_2dp": [f"{value:.2f}" for value in oof_scores],
            "predicted_label_at_zero": predictions,
        }
    )
    return pd.DataFrame(fold_rows), prediction_table, oof_scores


def fit_full_grid(
    features: pd.DataFrame, labels: np.ndarray
) -> tuple[GridSearchCV, pd.DataFrame]:
    """Run the reported full-data grid search and return its validation curve."""
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
    search = GridSearchCV(
        make_pipeline(),
        {"svc__C": C_GRID},
        scoring="roc_auc",
        cv=cv,
        n_jobs=-1,
        refit=True,
        return_train_score=True,
    )
    search.fit(features, labels)
    results = pd.DataFrame(search.cv_results_)
    grid_table = pd.DataFrame(
        {
            "C_full_precision": results["param_svc__C"].astype(float),
            "C_display_2dp": [f"{float(value):.2f}" for value in results["param_svc__C"]],
            "mean_train_AUROC": results["mean_train_score"],
            "std_train_AUROC": results["std_train_score"],
            "mean_validation_AUROC": results["mean_test_score"],
            "std_validation_AUROC": results["std_test_score"],
            "rank_validation_AUROC": results["rank_test_score"].astype(int),
        }
    ).sort_values("C_full_precision")
```

```
    return search, grid_table


def audit_final_model(
    search: GridSearchCV,
    features: pd.DataFrame,
    labels: np.ndarray,
    patient_ids: pd.Series,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    """Compute primal, dual, KKT, slack, and support-vector diagnostics."""
    best = search.best_estimator_
    transformed = best[:-1].transform(features)
    svc = best.named_steps["svc"]
    weights = svc.coef_[0]
    intercept = float(svc.intercept_[0])
    scores = transformed @ weights + intercept
    signed_margins = (2.0 * labels - 1.0) * scores
    slacks = np.maximum(0.0, 1.0 - signed_margins)
    best_c = float(search.best_params_["svc__C"])

    support_indices = svc.support_
    support_alpha = np.abs(svc.dual_coef_[0])
    support_signed_alpha = svc.dual_coef_[0]
    alpha = np.zeros(len(labels))
    alpha[support_indices] = support_alpha
    signed_labels = 2.0 * labels - 1.0

    primal_objective = float(0.5 * weights @ weights + best_c * slacks.sum())
    dual_objective = float(alpha.sum() - 0.5 * weights @ weights)
    reconstruction_residual = float(
        np.max(
            np.abs(
                weights
                - support_signed_alpha @ transformed[support_indices]
            )
        )
    )
    equality_residual = float(abs(alpha @ signed_labels))
    primal_constraint_residual = signed_margins - 1.0 + slacks
    margin_complementarity_residual = float(
        np.max(np.abs(alpha * (1.0 - slacks - signed_margins)))
    )
    slack_complementarity_residual = float(
        np.max(np.abs((best_c - alpha) * slacks))
    )

    if float(np.min(primal_constraint_residual)) < -1e-7:
        raise RuntimeError("Final model violates a soft-margin primal constraint.")
    if float(np.max(alpha)) > best_c + 1e-7:
        raise RuntimeError("Final model violates the dual upper bound alpha <= C.")
    if primal_objective - dual_objective > 1e-6:
        raise RuntimeError("Final primal-dual gap exceeds numerical tolerance.")

    support_table = pd.DataFrame(
        {
            "observation_number": support_indices + 1,
            "PatientID": patient_ids.iloc[support_indices].to_numpy(),
            "class": np.where(labels[support_indices] == 1, "UPA", "BPA"),
            "alpha_full_precision": support_alpha,
            "alpha_display_2dp": [f"{value:.2f}" for value in support_alpha],
            "decision_score_full_precision": scores[support_indices],
            "decision_score_display_2dp": [f"{value:.2f}" for value in scores[support_indices]],
            "signed_margin_full_precision": signed_margins[support_indices],
            "signed_margin_display_2dp": [f"{value:.2f}" for value in signed_margins[support_indices]],
            "slack_full_precision": slacks[support_indices],
            "slack_display_2dp": [f"{value:.2f}" for value in slacks[support_indices]],
            "at_upper_bound": np.isclose(support_alpha, best_c, atol=1e-5),
        }
    )

    audit_rows = [
        ("best_C", best_c, f"{best_c:.2f}"),
        ("full_grid_best_mean_validation_AUROC", float(search.best_score_), f"{search.best_score_:.2f}"),
```

```
        ("number_of_support_vectors", float(len(support_indices)), f"{len(support_indices)}"),
        ("number_with_positive_slack", float(np.count_nonzero(slacks > 1e-7)), f"{np.count_nonzero(slacks >
            1e-7)}"),
        ("sum_slack", float(slacks.sum()), f"{slacks.sum():.2f}"),
        ("weight_squared_norm", float(weights @ weights), f"{weights @ weights:.2f}"),
        ("intercept", intercept, f"{intercept:.2f}"),
        ("primal_objective", primal_objective, f"{primal_objective:.2f}"),
        ("dual_objective", dual_objective, f"{dual_objective:.2f}"),
        ("primal_dual_gap", primal_objective - dual_objective, f"{primal_objective - dual_objective:.2e}"),
        ("dual_equality_residual", equality_residual, f"{equality_residual:.2e}"),
        ("weight_reconstruction_residual", reconstruction_residual, f"{reconstruction_residual:.2e}"),
        ("margin_complementarity_residual", margin_complementarity_residual, f"{
            margin_complementarity_residual:.2e}"),
        ("slack_complementarity_residual", slack_complementarity_residual, f"{
            slack_complementarity_residual:.2e}"),
        ("minimum_primal_constraint_residual", float(np.min(primal_constraint_residual)), f"{float(np.min(
            primal_constraint_residual)):.2e}"),
    ]
    audit = pd.DataFrame(
        audit_rows,
        columns=["metric", "full_precision_value", "display_value"],
    )
    coefficient_table = pd.DataFrame(
        {
            "parameter": [*FEATURES, "Intercept"],
            "coefficient_full_precision": [*weights, intercept],
            "coefficient_display_2dp": [
                *[f"{value:.2f}" for value in weights],
                f"{intercept:.2f}",
            ],
        }
    )
    return audit, support_table, coefficient_table


def save_latex_tables(
    nested: pd.DataFrame,
    audit: pd.DataFrame,
    coefficients: pd.DataFrame,
    nested_metrics: dict[str, float],
) -> None:
    """Create report tables from the same values written to CSV."""
    TABLE_DIR.mkdir(parents=True, exist_ok=True)
    metric = dict(zip(audit["metric"], audit["display_value"]))
    summary_rows = [
        ("Selected $C$ from full-data grid search", metric["best_C"]),
        ("Mean inner-CV AUROC at selected $C$", metric["full_grid_best_mean_validation_AUROC"]),
        ("Nested out-of-fold AUROC", f"{nested_metrics['oof_auc']:.2f}"),
        ("Nested out-of-fold accuracy", f"{nested_metrics['accuracy']:.2f}"),
        ("Nested sensitivity", f"{nested_metrics['sensitivity']:.2f}"),
        ("Nested specificity", f"{nested_metrics['specificity']:.2f}"),
        ("Full-data support vectors", metric["number_of_support_vectors"]),
        (r"Full-data observations with $\xi_i>0$", metric["number_with_positive_slack"]),
        (r"Full-data $\sum_i\xi_i$", metric["sum_slack"]),
        ("Full-data primal objective", metric["primal_objective"]),
        ("Full-data dual objective", metric["dual_objective"]),
    ]
    summary_table = "\n".join(
        [r"\begin{tabular}{lr}", r"\toprule", r"Quantity & Value \\", r"\midrule"]
        + [f"{name} & {value} \\\\" for name, value in summary_rows]
        + [r"\bottomrule", r"\end{tabular}", ""]
    )
    (TABLE_DIR / "soft_margin_summary_table.tex").write_text(
        summary_table, encoding="utf-8"
    )

    fold_rows = []
    for row in nested.itertuples(index=False):
        fold_rows.append(
            f"{row.outer_fold} & {row.n_train} & {row.n_test} & "
            f"{float(row.best_C_full_precision):.2f} & "
            f"{row.inner_best_mean_AUROC:.2f} & {row.outer_test_AUROC:.2f} \\\\"
        )
```

```
    fold_table = "\n".join(
        [
            r"\begin{tabular}{rrrrrr}",
            r"\toprule",
            r"Outer fold & Train & Test & Selected $C$ & Inner AUROC & Outer AUROC \\",
            r"\midrule",
            *fold_rows,
            r"\bottomrule",
            r"\end{tabular}",
            "",
        ]
    )
    (TABLE_DIR / "soft_margin_nested_cv_table.tex").write_text(
        fold_table, encoding="utf-8"
    )

    coefficient_rows = []
    for left_index in range(5):
        left = coefficients.iloc[left_index]
        right = coefficients.iloc[left_index + 5]
        coefficient_rows.append(
            f"{left['parameter']} & {left['coefficient_display_2dp']} & "
            f"{right['parameter']} & {right['coefficient_display_2dp']} \\\\"
        )
    intercept = coefficients.iloc[-1]
    coefficient_rows.append(
        f"{intercept['parameter']} & {intercept['coefficient_display_2dp']} & & \\\\"
    )
    coefficient_table = "\n".join(
        [
            r"\begin{tabular}{lr@{\qquad}lr}",
            r"\toprule",
            r"Parameter & Estimate & Parameter & Estimate \\",
            r"\midrule",
            *coefficient_rows,
            r"\bottomrule",
            r"\end{tabular}",
            "",
        ]
    )
    (TABLE_DIR / "soft_margin_coefficients_table.tex").write_text(
        coefficient_table, encoding="utf-8"
    )


def save_figure(
    grid: pd.DataFrame,
    labels: np.ndarray,
    oof_scores: np.ndarray,
    best_c: float,
    oof_auc: float,
) -> None:
    """Save the C validation curve and nested out-of-fold ROC."""
    FIGURE_STEM.parent.mkdir(parents=True, exist_ok=True)
    fig, (ax_grid, ax_roc) = plt.subplots(
        1, 2, figsize=(10.6, 4.4), constrained_layout=True
    )
    x = grid["C_full_precision"].to_numpy()
    mean = grid["mean_validation_AUROC"].to_numpy()
    std = grid["std_validation_AUROC"].to_numpy()
    ax_grid.semilogx(x, mean, color="#1F77B4", marker="o", linewidth=1.8)
    ax_grid.fill_between(x, mean - std, mean + std, color="#1F77B4", alpha=0.18)
    ax_grid.axvline(best_c, color="#D62728", linestyle="--", linewidth=1.4)
    ax_grid.set(
        xlabel="Penalty parameter $C$ (log scale)",
        ylabel="Mean five-fold validation AUROC",
        title="(a) Grid search for $C$",
        ylim=(0.45, 1.01),
    )
    ax_grid.yaxis.set_major_formatter(FormatStrFormatter("%.2f"))
    ax_grid.grid(alpha=0.22)
    ax_grid.text(
        0.97,
```

```
        0.05,
        f"selected $C={best_c:.2f}$",
        transform=ax_grid.transAxes,
        ha="right",
        bbox={"boxstyle": "round,pad=0.25", "facecolor": "white", "edgecolor": "#D1D5DB"},
    )

    fpr, tpr, _ = roc_curve(labels, oof_scores)
    ax_roc.plot(fpr, tpr, color="#1F77B4", linewidth=2.0, label=f"AUROC = {oof_auc:.2f}")
    ax_roc.plot([0, 1], [0, 1], color="#6B7280", linestyle=":", linewidth=1.2, label="Chance")
    ax_roc.set(
        xlabel="False-positive rate",
        ylabel="True-positive rate",
        title="(b) Nested out-of-fold ROC",
        xlim=(-0.01, 1.01),
        ylim=(-0.01, 1.01),
    )
    ax_roc.xaxis.set_major_formatter(FormatStrFormatter("%.2f"))
    ax_roc.yaxis.set_major_formatter(FormatStrFormatter("%.2f"))
    ax_roc.set_aspect("equal", adjustable="box")
    ax_roc.grid(alpha=0.22)
    ax_roc.legend(loc="lower right")
    fig.suptitle("Linear soft-margin SVM: model selection and validation", fontweight="bold")
    fig.savefig(FIGURE_STEM.with_suffix(".png"), dpi=300, facecolor="white")
    fig.savefig(FIGURE_STEM.with_suffix(".pdf"), facecolor="white")
    plt.close(fig)


def main() -> None:
    frame = pd.read_csv(DATA_PATH)
    features = frame[FEATURES].apply(pd.to_numeric, errors="coerce")
    labels = frame["Model 1 Target"].eq("UPA").astype(int).to_numpy()

    nested, predictions, oof_scores = nested_grid_search(features, labels)
    search, grid = fit_full_grid(features, labels)
    audit, support, coefficients = audit_final_model(
        search, features, labels, frame["PatientID"]
    )

    tn, fp, fn, tp = confusion_matrix(labels, oof_scores >= 0.0).ravel()
    nested_metrics = {
        "oof_auc": float(roc_auc_score(labels, oof_scores)),
        "accuracy": float(accuracy_score(labels, oof_scores >= 0.0)),
        "sensitivity": float(tp / (tp + fn)),
        "specificity": float(tn / (tn + fp)),
    }

    CSV_DIR.mkdir(parents=True, exist_ok=True)
    nested.to_csv(CSV_DIR / "soft_margin_nested_cv.csv", index=False, float_format="%.15g")
    predictions.insert(1, "PatientID", frame["PatientID"])
    predictions.to_csv(CSV_DIR / "soft_margin_oof_predictions.csv", index=False, float_format="%.15g")
    grid.to_csv(CSV_DIR / "soft_margin_grid_search.csv", index=False, float_format="%.15g")
    audit.to_csv(CSV_DIR / "soft_margin_solution_audit.csv", index=False, float_format="%.15g")
    support.to_csv(CSV_DIR / "soft_margin_support_vectors.csv", index=False, float_format="%.15g")
    coefficients.to_csv(
        CSV_DIR / "soft_margin_coefficients.csv", index=False, float_format="%.15g"
    )
    pd.DataFrame([nested_metrics]).to_csv(
        CSV_DIR / "soft_margin_nested_metrics.csv", index=False, float_format="%.15g"
    )

    save_latex_tables(nested, audit, coefficients, nested_metrics)
    save_figure(
        grid,
        labels,
        oof_scores,
        float(search.best_params_["svc__C"]),
        nested_metrics["oof_auc"],
    )

    print(f"Best full-data C: {float(search.best_params_['svc__C']):.12g}")
    print(f"Full-grid mean validation AUROC: {float(search.best_score_):.6f}")
    print(f"Nested out-of-fold AUROC: {nested_metrics['oof_auc']:.6f}")
```

```
    print(audit.to_string(index=False))


if __name__ == "__main__":
    main()
```

E RBF Kernel Grid Search and Overfitting Audit Code

The implementation below jointly tunes C and γ inside nested cross-validation, evaluates train–validation gaps, compares RBF and linear outer-fold performance, and generates the Chapter 6 artifacts.

```
#!/usr/bin/env python3
"""Tune and audit an RBF-kernel soft-margin SVM with nested CV."""

from __future__ import annotations

import os
from pathlib import Path

os.environ.setdefault("MPLCONFIGDIR", "/tmp/mpl-cache")

import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt
import _pubstyle; _pubstyle.apply()
from matplotlib.colors import TwoSlopeNorm
from matplotlib.ticker import FormatStrFormatter
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.metrics import accuracy_score, confusion_matrix, roc_auc_score, roc_curve
from sklearn.metrics.pairwise import rbf_kernel
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import FunctionTransformer, StandardScaler
from sklearn.svm import SVC

from linear_hard_margin_svm import FEATURES
from linear_soft_margin_svm import log_selected_columns


ROOT = Path(__file__).resolve().parents[1]
DATA_PATH = ROOT / "data" / "cgh_pa_dataset.csv"
CSV_DIR = ROOT / "output" / "csv"
TABLE_DIR = ROOT / "output" / "tables"
FIGURE_DIR = ROOT / "output" / "figures"

C_GRID = 10.0 ** np.arange(-2.0, 2.01, 0.5)
GAMMA_GRID = 10.0 ** np.arange(-3.0, 1.01, 0.5)
RANDOM_STATE = 0

plt.rcParams.update({"pdf.fonttype": 42, "ps.fonttype": 42})


def display_parameter(value: float) -> str:
    """Use two decimals, switching to scientific notation below 0.01."""
    return f"{value:.2e}" if value < 0.01 else f"{value:.2f}"


def make_pipeline() -> Pipeline:
    """Return a leakage-safe RBF-kernel SVM pipeline."""
    return Pipeline(
        [
            ("log_transform", FunctionTransformer(log_selected_columns, validate=False)),
            ("median_imputer", SimpleImputer(strategy="median")),
            ("standard_scaler", StandardScaler()),
```

```
            ("svc", SVC(kernel="rbf", C=1.0, gamma="scale", tol=1e-7, max_iter=-1)),
        ]
    )


def nested_search(
    features: pd.DataFrame, labels: np.ndarray
) -> tuple[pd.DataFrame, pd.DataFrame]:
    """Tune C and gamma inside each outer fold and return OOF predictions."""
    outer = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
    rows: list[dict[str, float | int]] = []
    predictions = np.full(len(labels), np.nan)
    fold_assignment = np.zeros(len(labels), dtype=int)

    for fold, (train_indices, test_indices) in enumerate(
        outer.split(features, labels), start=1
    ):
        inner = StratifiedKFold(
            n_splits=5, shuffle=True, random_state=RANDOM_STATE + fold
        )
        search = GridSearchCV(
            make_pipeline(),
            {"svc__C": C_GRID, "svc__gamma": GAMMA_GRID},
            scoring="roc_auc",
            cv=inner,
            n_jobs=-1,
            refit=True,
            return_train_score=True,
        )
        search.fit(features.iloc[train_indices], labels[train_indices])
        scores = search.decision_function(features.iloc[test_indices])
        predictions[test_indices] = scores
        fold_assignment[test_indices] = fold
        best_index = search.best_index_
        train_auc = float(search.cv_results_["mean_train_score"][best_index])
        validation_auc = float(search.best_score_)
        best_c = float(search.best_params_["svc__C"])
        best_gamma = float(search.best_params_["svc__gamma"])
        rows.append(
            {
                "outer_fold": fold,
                "n_train": len(train_indices),
                "n_test": len(test_indices),
                "best_C_full_precision": best_c,
                "best_C_display_2dp": display_parameter(best_c),
                "best_gamma_full_precision": best_gamma,
                "best_gamma_display_2dp": display_parameter(best_gamma),
                "inner_mean_train_AUROC": train_auc,
                "inner_mean_validation_AUROC": validation_auc,
                "inner_train_validation_gap": train_auc - validation_auc,
                "outer_test_AUROC": float(
                    roc_auc_score(labels[test_indices], scores)
                ),
            }
        )

    if np.isnan(predictions).any() or np.any(fold_assignment == 0):
        raise RuntimeError("Nested RBF search did not score every observation.")
    prediction_table = pd.DataFrame(
        {
            "observation_number": np.arange(1, len(labels) + 1),
            "outer_fold": fold_assignment,
            "true_label": labels,
            "oof_decision_score_full_precision": predictions,
            "oof_decision_score_display_2dp": [f"{value:.2f}" for value in predictions],
            "predicted_label_at_zero": (predictions >= 0.0).astype(int),
        }
    )
    return pd.DataFrame(rows), prediction_table


def full_grid_search(
    features: pd.DataFrame, labels: np.ndarray
```

```
) -> tuple[GridSearchCV, pd.DataFrame]:
    """Tune the reported final RBF model on the complete dataset."""
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
    search = GridSearchCV(
        make_pipeline(),
        {"svc__C": C_GRID, "svc__gamma": GAMMA_GRID},
        scoring="roc_auc",
        cv=cv,
        n_jobs=-1,
        refit=True,
        return_train_score=True,
    )
    search.fit(features, labels)
    results = pd.DataFrame(search.cv_results_)
    table = pd.DataFrame(
        {
            "C_full_precision": results["param_svc__C"].astype(float),
            "C_display_2dp": [display_parameter(float(value)) for value in results["param_svc__C"]],
            "gamma_full_precision": results["param_svc__gamma"].astype(float),
            "gamma_display_2dp": [display_parameter(float(value)) for value in results["param_svc__gamma"]],

            "mean_train_AUROC": results["mean_train_score"],
            "std_train_AUROC": results["std_train_score"],
            "mean_validation_AUROC": results["mean_test_score"],
            "std_validation_AUROC": results["std_test_score"],
            "train_validation_gap": results["mean_train_score"] - results["mean_test_score"],
            "rank_validation_AUROC": results["rank_test_score"].astype(int),
        }
    ).sort_values(["C_full_precision", "gamma_full_precision"])
    return search, table


def audit_model(
    search: GridSearchCV,
    features: pd.DataFrame,
    labels: np.ndarray,
    patient_ids: pd.Series,
) -> tuple[pd.DataFrame, pd.DataFrame]:
    """Audit RBF primal/dual values in the induced feature space."""
    best = search.best_estimator_
    transformed = best[:-1].transform(features)
    svc = best.named_steps["svc"]
    support_indices = svc.support_
    signed_labels = 2.0 * labels - 1.0
    scores = best.decision_function(features)
    margins = signed_labels * scores
    slacks = np.maximum(0.0, 1.0 - margins)
    alpha_support = np.abs(svc.dual_coef_[0])
    signed_alpha_support = svc.dual_coef_[0]
    alpha = np.zeros(len(labels))
    alpha[support_indices] = alpha_support
    best_c = float(search.best_params_["svc__C"])
    best_gamma = float(search.best_params_["svc__gamma"])

    kernel_matrix = rbf_kernel(
        transformed[support_indices],
        transformed[support_indices],
        gamma=best_gamma,
    )
    rkhs_norm_squared = float(
        signed_alpha_support @ kernel_matrix @ signed_alpha_support
    )
    primal = float(0.5 * rkhs_norm_squared + best_c * slacks.sum())
    dual = float(alpha_support.sum() - 0.5 * rkhs_norm_squared)
    equality_residual = float(abs(alpha @ signed_labels))
    margin_complementarity = float(
        np.max(np.abs(alpha * (1.0 - slacks - margins)))
    )
    slack_complementarity = float(
        np.max(np.abs((best_c - alpha) * slacks))
    )

    if primal - dual > 1e-5:
```

```
        raise RuntimeError("RBF primal-dual gap exceeds numerical tolerance.")
    if alpha.max() > best_c + 1e-7:
        raise RuntimeError("RBF solution violates alpha <= C.")

    best_index = search.best_index_
    train_auc = float(search.cv_results_["mean_train_score"][best_index])
    validation_auc = float(search.best_score_)
    audit = pd.DataFrame(
        [
            ("best_C", best_c, display_parameter(best_c)),
            ("best_gamma", best_gamma, display_parameter(best_gamma)),
            ("full_grid_mean_train_AUROC", train_auc, f"{train_auc:.2f}"),
            ("full_grid_mean_validation_AUROC", validation_auc, f"{validation_auc:.2f}"),
            ("full_grid_train_validation_gap", train_auc - validation_auc, f"{train_auc - validation_auc:.2
                f}"),
            ("number_of_support_vectors", float(len(support_indices)), str(len(support_indices))),
            ("number_with_positive_slack", float(np.count_nonzero(slacks > 1e-7)), str(np.count_nonzero(
                slacks > 1e-7))),
            ("sum_slack", float(slacks.sum()), f"{slacks.sum():.2f}"),
            ("rkhs_norm_squared", rkhs_norm_squared, f"{rkhs_norm_squared:.2f}"),
            ("primal_objective", primal, f"{primal:.2f}"),
            ("dual_objective", dual, f"{dual:.2f}"),
            ("primal_dual_gap", primal - dual, f"{primal - dual:.2e}"),
            ("dual_equality_residual", equality_residual, f"{equality_residual:.2e}"),
            ("margin_complementarity_residual", margin_complementarity, f"{margin_complementarity:.2e}"),
            ("slack_complementarity_residual", slack_complementarity, f"{slack_complementarity:.2e}"),
        ],
        columns=["metric", "full_precision_value", "display_value"],
    )
    support = pd.DataFrame(
        {
            "observation_number": support_indices + 1,
            "PatientID": patient_ids.iloc[support_indices].to_numpy(),
            "class": np.where(labels[support_indices] == 1, "UPA", "BPA"),
            "alpha_full_precision": alpha_support,
            "alpha_display_2dp": [f"{value:.2f}" for value in alpha_support],
            "signed_margin_full_precision": margins[support_indices],
            "signed_margin_display_2dp": [f"{value:.2f}" for value in margins[support_indices]],
            "slack_full_precision": slacks[support_indices],
            "slack_display_2dp": [f"{value:.2f}" for value in slacks[support_indices]],
            "at_upper_bound": np.isclose(alpha_support, best_c, atol=1e-5),
        }
    )
    return audit, support


def compute_comparison(
    nested: pd.DataFrame, predictions: pd.DataFrame
) -> tuple[pd.DataFrame, dict[str, float]]:
    """Compare RBF nested performance with the existing linear nested audit."""
    linear_nested = pd.read_csv(CSV_DIR / "soft_margin_nested_cv.csv")
    linear_metrics = pd.read_csv(CSV_DIR / "soft_margin_nested_metrics.csv").iloc[0]
    y = predictions["true_label"].to_numpy()
    scores = predictions["oof_decision_score_full_precision"].to_numpy()
    tn, fp, fn, tp = confusion_matrix(y, scores >= 0.0).ravel()
    rbf_metrics = {
        "pooled_oof_AUROC": float(roc_auc_score(y, scores)),
        "mean_outer_AUROC": float(nested["outer_test_AUROC"].mean()),
        "std_outer_AUROC": float(nested["outer_test_AUROC"].std(ddof=1)),
        "accuracy": float(accuracy_score(y, scores >= 0.0)),
        "sensitivity": float(tp / (tp + fn)),
        "specificity": float(tn / (tn + fp)),
    }
    linear_mean = float(linear_nested["outer_test_AUROC"].mean())
    linear_std = float(linear_nested["outer_test_AUROC"].std(ddof=1))
    comparison = pd.DataFrame(
        [
            {
                "model": "Linear soft margin",
                "tuned_parameters": "C",
                "mean_outer_AUROC": linear_mean,
                "std_outer_AUROC": linear_std,
                "pooled_oof_AUROC": float(linear_metrics["oof_auc"]),
```

```
            "accuracy": float(linear_metrics["accuracy"]),
            "sensitivity": float(linear_metrics["sensitivity"]),
            "specificity": float(linear_metrics["specificity"]),
        },
        {
            "model": "RBF kernel",
            "tuned_parameters": "C and gamma",
            "mean_outer_AUROC": rbf_metrics["mean_outer_AUROC"],
            "std_outer_AUROC": rbf_metrics["std_outer_AUROC"],
            "pooled_oof_AUROC": rbf_metrics["pooled_oof_AUROC"],
            "accuracy": rbf_metrics["accuracy"],
            "sensitivity": rbf_metrics["sensitivity"],
            "specificity": rbf_metrics["specificity"],
        },
    ]
)
return comparison, rbf_metrics


def save_tables(
    nested: pd.DataFrame,
    audit: pd.DataFrame,
    comparison: pd.DataFrame,
) -> None:
    """Write compact LaTeX tables generated from CSV-bound results."""
    TABLE_DIR.mkdir(parents=True, exist_ok=True)
    nested_rows = []
    for row in nested.itertuples(index=False):
        nested_rows.append(
            f"{row.outer_fold} & {display_parameter(row.best_C_full_precision)} & "
            f"{display_parameter(row.best_gamma_full_precision)} & "
            f"{row.inner_mean_train_AUROC:.2f} & {row.inner_mean_validation_AUROC:.2f} & "
            f"{row.outer_test_AUROC:.2f} \\\\"
        )
    nested_table = "\n".join(
        [
            r"\begin{tabular}{rrrrrr}", r"\toprule",
            r"Fold & $C$ & $\gamma$ & Inner train & Inner validation & Outer AUROC \\",
            r"\midrule", *nested_rows, r"\bottomrule", r"\end{tabular}", "",
        ]
    )
    (TABLE_DIR / "rbf_nested_cv_table.tex").write_text(nested_table, encoding="utf-8")

    metric = dict(zip(audit["metric"], audit["display_value"]))
    rows = [
        ("Selected $C$", metric["best_C"]),
        (r"Selected $\gamma$", metric["best_gamma"]),
        ("Mean training AUROC", metric["full_grid_mean_train_AUROC"]),
        ("Mean validation AUROC", metric["full_grid_mean_validation_AUROC"]),
        ("Train--validation gap", metric["full_grid_train_validation_gap"]),
        ("Largest gap anywhere on grid", metric["maximum_grid_train_validation_gap"]),
        ("Mean nested outer AUROC", metric["mean_nested_outer_AUROC"]),
        ("Support vectors", metric["number_of_support_vectors"]),
        (r"Observations with $\xi_i>0$", metric["number_with_positive_slack"]),
        ("Primal objective", metric["primal_objective"]),
        ("Dual objective", metric["dual_objective"]),
    ]
    summary = "\n".join(
        [r"\begin{tabular}{lr}", r"\toprule", r"Quantity & Value \\", r"\midrule"]
        + [f"{name} & {value} \\\\" for name, value in rows]
        + [r"\bottomrule", r"\end{tabular}", ""]
    )
    (TABLE_DIR / "rbf_summary_table.tex").write_text(summary, encoding="utf-8")

    comparison_rows = []
    for row in comparison.itertuples(index=False):
        comparison_rows.append(
            f"{row.model} & {row.mean_outer_AUROC:.2f} & {row.std_outer_AUROC:.2f} & "
            f"{row.pooled_oof_AUROC:.2f} & {row.accuracy:.2f} & "
            f"{row.sensitivity:.2f} & {row.specificity:.2f} \\\\"
        )
    comparison_table = "\n".join(
        [
```

```
            r"\begin{tabular}{lrrrrrr}", r"\toprule",
            r"Model & Mean outer AUC & SD & Pooled AUC & Accuracy & Sensitivity & Specificity \\",
            r"\midrule", *comparison_rows, r"\bottomrule", r"\end{tabular}", "",
        ]
    )
    (TABLE_DIR / "kernel_comparison_table.tex").write_text(
        comparison_table, encoding="utf-8"
    )


def save_grid_figure(grid: pd.DataFrame, best_c: float, best_gamma: float) -> None:
    """Plot validation AUROC and train-validation gap over C and gamma."""
    FIGURE_DIR.mkdir(parents=True, exist_ok=True)
    validation = grid.pivot(index="gamma_full_precision", columns="C_full_precision", values="
        mean_validation_AUROC")
    gap = grid.pivot(index="gamma_full_precision", columns="C_full_precision", values="
        train_validation_gap")
    fig, (ax_auc, ax_gap) = plt.subplots(1, 2, figsize=(11.2, 4.6), constrained_layout=True)
    extent = (-0.5, len(C_GRID) - 0.5, -0.5, len(GAMMA_GRID) - 0.5)
    auc_image = ax_auc.imshow(validation.to_numpy(), origin="lower", aspect="auto", cmap="viridis", vmin
        =0.5, vmax=1.0, extent=extent)
    gap_image = ax_gap.imshow(gap.to_numpy(), origin="lower", aspect="auto", cmap="magma", norm=
        TwoSlopeNorm(vmin=0.0, vcenter=0.10, vmax=max(0.20, float(gap.max().max()))), extent=extent)
    best_x = int(np.where(np.isclose(C_GRID, best_c))[0][0])
    best_y = int(np.where(np.isclose(GAMMA_GRID, best_gamma))[0][0])
    for ax in (ax_auc, ax_gap):
        ax.scatter(best_x, best_y, marker="*", s=150, color="white", edgecolor="black", linewidth=0.8,
            zorder=4)
        ax.set_xticks(np.arange(len(C_GRID)), [display_parameter(v) for v in C_GRID], rotation=45, ha="
            right")
        ax.set_yticks(np.arange(len(GAMMA_GRID)), [display_parameter(v) for v in GAMMA_GRID])
        ax.set_xlabel("Penalty $C$")
        ax.set_ylabel(r"RBF width $\gamma$")
    ax_auc.set_title("(a) Mean validation AUROC")
    ax_gap.set_title("(b) Train--validation AUROC gap")
    fig.colorbar(auc_image, ax=ax_auc, fraction=0.046, pad=0.03, format="%.2f")
    fig.colorbar(gap_image, ax=ax_gap, fraction=0.046, pad=0.03, format="%.2f")
    fig.suptitle("RBF grid search and overfitting diagnostic", fontweight="bold")
    stem = FIGURE_DIR / "rbf_kernel_grid_heatmaps"
    fig.savefig(stem.with_suffix(".png"), dpi=300, facecolor="white")
    fig.savefig(stem.with_suffix(".pdf"), facecolor="white")
    plt.close(fig)


def mean_roc(
    labels: np.ndarray, scores: np.ndarray, folds: np.ndarray
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """Interpolate five fold-specific ROC curves on a common FPR grid."""
    mean_fpr = np.linspace(0.0, 1.0, 201)
    curves = []
    for fold in sorted(np.unique(folds)):
        mask = folds == fold
        fpr, tpr, _ = roc_curve(labels[mask], scores[mask])
        interpolated = np.interp(mean_fpr, fpr, tpr)
        interpolated[0] = 0.0
        interpolated[-1] = 1.0
        curves.append(interpolated)
    matrix = np.vstack(curves)
    return mean_fpr, matrix.mean(axis=0), matrix.std(axis=0, ddof=1)


def save_roc_figure(predictions: pd.DataFrame, nested: pd.DataFrame) -> None:
    """Compare mean outer-fold ROC curves for linear and RBF models."""
    linear_predictions = pd.read_csv(CSV_DIR / "soft_margin_oof_predictions.csv")
    labels = predictions["true_label"].to_numpy()
    folds = predictions["outer_fold"].to_numpy()
    rbf_fpr, rbf_mean, rbf_std = mean_roc(
        labels, predictions["oof_decision_score_full_precision"].to_numpy(), folds
    )
    linear_fpr, linear_mean, linear_std = mean_roc(
        labels, linear_predictions["oof_decision_score_full_precision"].to_numpy(), folds
    )
    linear_nested = pd.read_csv(CSV_DIR / "soft_margin_nested_cv.csv")
```

```
    fig, ax = plt.subplots(figsize=(6.4, 5.2), constrained_layout=True)
    ax.plot(linear_fpr, linear_mean, color="#1F77B4", linewidth=2.0, label=f"Linear: mean AUC {
        linear_nested.outer_test_AUROC.mean():.2f}")
    ax.fill_between(linear_fpr, np.clip(linear_mean-linear_std, 0, 1), np.clip(linear_mean+linear_std, 0,
        1), color="#1F77B4", alpha=0.12)
    ax.plot(rbf_fpr, rbf_mean, color="#D62728", linewidth=2.0, label=f"RBF: mean AUC {nested.
        outer_test_AUROC.mean():.2f}")
    ax.fill_between(rbf_fpr, np.clip(rbf_mean-rbf_std, 0, 1), np.clip(rbf_mean+rbf_std, 0, 1), color="#
        D62728", alpha=0.12)
    ax.plot([0, 1], [0, 1], color="#6B7280", linestyle=":", linewidth=1.2, label="Chance")
    ax.set(xlabel="False-positive rate", ylabel="True-positive rate", title="Nested outer-fold ROC: linear
         versus RBF", xlim=(-0.01, 1.01), ylim=(-0.01, 1.01))
    ax.xaxis.set_major_formatter(FormatStrFormatter("%.2f"))
    ax.yaxis.set_major_formatter(FormatStrFormatter("%.2f"))
    ax.set_aspect("equal", adjustable="box")
    ax.grid(alpha=0.22)
    ax.legend(loc="lower right")
    stem = FIGURE_DIR / "linear_vs_rbf_nested_roc"
    fig.savefig(stem.with_suffix(".png"), dpi=300, facecolor="white")
    fig.savefig(stem.with_suffix(".pdf"), facecolor="white")
    plt.close(fig)


def mark_pareto_frontier(
    frame: pd.DataFrame,
    score_column: str,
    gap_column: str,
) -> np.ndarray:
    """Mark points that maximize score while minimizing overfitting gap."""
    scores = frame[score_column].to_numpy(dtype=float)
    gaps = frame[gap_column].to_numpy(dtype=float)
    frontier = np.ones(len(frame), dtype=bool)
    for index in range(len(frame)):
        weakly_better = (scores >= scores[index]) & (gaps <= gaps[index])
        strictly_better = (scores > scores[index]) | (gaps < gaps[index])
        if np.any(weakly_better & strictly_better):
            frontier[index] = False
    return frontier


def build_pareto_table(rbf_grid: pd.DataFrame) -> pd.DataFrame:
    """Combine the linear and RBF grids and flag non-dominated settings."""
    linear = pd.read_csv(CSV_DIR / "soft_margin_grid_search.csv").copy()
    linear["train_validation_gap"] = (
        linear["mean_train_AUROC"] - linear["mean_validation_AUROC"]
    )
    linear["is_pareto_optimal"] = mark_pareto_frontier(
        linear, "mean_validation_AUROC", "train_validation_gap"
    )
    linear_table = pd.DataFrame(
        {
            "model": "Linear soft margin",
            "C_full_precision": linear["C_full_precision"],
            "gamma_full_precision": np.nan,
            "mean_train_AUROC": linear["mean_train_AUROC"],
            "mean_validation_AUROC": linear["mean_validation_AUROC"],
            "train_validation_gap": linear["train_validation_gap"],
            "is_pareto_optimal": linear["is_pareto_optimal"],
        }
    )

    rbf = rbf_grid.copy()
    rbf["is_pareto_optimal"] = mark_pareto_frontier(
        rbf, "mean_validation_AUROC", "train_validation_gap"
    )
    rbf_table = pd.DataFrame(
        {
            "model": "RBF kernel",
            "C_full_precision": rbf["C_full_precision"],
            "gamma_full_precision": rbf["gamma_full_precision"],
            "mean_train_AUROC": rbf["mean_train_AUROC"],
            "mean_validation_AUROC": rbf["mean_validation_AUROC"],
            "train_validation_gap": rbf["train_validation_gap"],
```

```
            "is_pareto_optimal": rbf["is_pareto_optimal"],
        }
    )
    table = pd.concat([linear_table, rbf_table], ignore_index=True)
    table["C_display_2dp"] = table["C_full_precision"].map(display_parameter)
    table["gamma_display_2dp"] = table["gamma_full_precision"].map(
        lambda value: "--" if pd.isna(value) else display_parameter(float(value))
    )
    table["validation_AUROC_display_2dp"] = table[
        "mean_validation_AUROC"
    ].map(lambda value: f"{value:.2f}")
    table["gap_display_2dp"] = table["train_validation_gap"].map(
        lambda value: f"{value:.2f}"
    )
    return table


def save_pareto_figure(table: pd.DataFrame) -> None:
    """Plot validation AUROC versus overfitting gap for both model grids."""
    FIGURE_DIR.mkdir(parents=True, exist_ok=True)
    fig, (ax_linear, ax_rbf) = plt.subplots(
        1, 2, figsize=(11.2, 4.7), constrained_layout=True
    )

    linear = table[table["model"].eq("Linear soft margin")].copy()
    linear_front = linear[linear["is_pareto_optimal"]]
    linear_scatter = ax_linear.scatter(
        linear["train_validation_gap"],
        linear["mean_validation_AUROC"],
        c=np.log10(linear["C_full_precision"]),
        cmap="viridis",
        s=58,
        alpha=0.75,
        edgecolor="white",
        linewidth=0.5,
    )
    ax_linear.scatter(
        linear_front["train_validation_gap"],
        linear_front["mean_validation_AUROC"],
        marker="*",
        s=230,
        color="#D62728",
        edgecolor="black",
        linewidth=0.8,
        label="Pareto-optimal",
        zorder=5,
    )
    ax_linear.annotate(
        "$C=0.01$",
        xy=(
            float(linear_front["train_validation_gap"].iloc[0]),
            float(linear_front["mean_validation_AUROC"].iloc[0]),
        ),
        xytext=(10, -22),
        textcoords="offset points",
        fontsize=8.5,
    )
    ax_linear.set_title("(a) Linear soft-margin grid")
    fig.colorbar(
        linear_scatter,
        ax=ax_linear,
        fraction=0.046,
        pad=0.03,
        label=r"$\log_{10}C$",
        format="%.2f",
    )

    rbf = table[table["model"].eq("RBF kernel")].copy()
    rbf_front = rbf[rbf["is_pareto_optimal"]]
    log_c = np.log10(rbf["C_full_precision"].to_numpy())
    sizes = 34.0 + 15.0 * (log_c - log_c.min())
    rbf_scatter = ax_rbf.scatter(
        rbf["train_validation_gap"],
```

```
        rbf["mean_validation_AUROC"],
        c=np.log10(rbf["gamma_full_precision"]),
        cmap="plasma",
        s=sizes,
        alpha=0.58,
        edgecolor="white",
        linewidth=0.35,
    )
    ax_rbf.scatter(
        rbf_front["train_validation_gap"],
        rbf_front["mean_validation_AUROC"],
        marker="*",
        s=230,
        color="#D62728",
        edgecolor="black",
        linewidth=0.8,
        label="Pareto-optimal",
        zorder=5,
    )
    ax_rbf.annotate(
        "Pareto plateau:\n4 values of $C$ at $\\gamma=0.00316$",
        xy=(
            float(rbf_front["train_validation_gap"].iloc[0]),
            float(rbf_front["mean_validation_AUROC"].iloc[0]),
        ),
        xytext=(0.52, 0.88),
        textcoords="axes fraction",
        fontsize=8.5,
        arrowprops={"arrowstyle": "->", "color": "#4B5563", "linewidth": 0.8},
        bbox={"boxstyle": "round,pad=0.25", "facecolor": "white", "edgecolor": "#D1D5DB", "alpha": 0.92},
    )
    ax_rbf.text(
        0.98,
        0.04,
        "marker size increases with $C$",
        transform=ax_rbf.transAxes,
        ha="right",
        fontsize=8.2,
        color="#4B5563",
    )
    ax_rbf.set_title(r"(b) RBF $C$--$\gamma$ grid")
    fig.colorbar(
        rbf_scatter,
        ax=ax_rbf,
        fraction=0.046,
        pad=0.03,
        label=r"$\log_{10}\gamma$",
        format="%.2f",
    )

    for ax in (ax_linear, ax_rbf):
        ax.set_xlabel("Train--validation AUROC gap (lower is better)")
        ax.set_ylabel("Mean validation AUROC (higher is better)")
        ax.xaxis.set_major_formatter(FormatStrFormatter("%.2f"))
        ax.yaxis.set_major_formatter(FormatStrFormatter("%.2f"))
        ax.grid(alpha=0.20)
        ax.legend(loc="lower left", fontsize=8.5)
    fig.suptitle(
        "Pareto frontier: validation performance versus overfitting",
        fontweight="bold",
    )
    stem = FIGURE_DIR / "svm_grid_pareto_frontier"
    fig.savefig(stem.with_suffix(".png"), dpi=300, facecolor="white")
    fig.savefig(stem.with_suffix(".pdf"), facecolor="white")
    plt.close(fig)


def main() -> None:
    frame = pd.read_csv(DATA_PATH)
    features = frame[FEATURES].apply(pd.to_numeric, errors="coerce")
    labels = frame["Model 1 Target"].eq("UPA").astype(int).to_numpy()
    nested, predictions = nested_search(features, labels)
    search, grid = full_grid_search(features, labels)
```

```
    audit, support = audit_model(search, features, labels, frame["PatientID"])
    comparison, metrics = compute_comparison(nested, predictions)
    worst = grid.loc[grid["train_validation_gap"].idxmax()]
    linear_outer_auc = float(comparison.loc[comparison["model"].eq("Linear soft margin"), "
        mean_outer_AUROC"].iat[0])
    extra_audit = pd.DataFrame(
        [
            ("mean_nested_outer_AUROC", float(nested["outer_test_AUROC"].mean()), f"{nested['
                outer_test_AUROC'].mean():.2f}"),
            ("std_nested_outer_AUROC", float(nested["outer_test_AUROC"].std(ddof=1)), f"{nested['
                outer_test_AUROC'].std(ddof=1):.2f}"),
            ("rbf_minus_linear_mean_outer_AUROC", float(nested["outer_test_AUROC"].mean()) -
                linear_outer_auc, f"{float(nested['outer_test_AUROC'].mean()) - linear_outer_auc:.2e}"),
            ("maximum_grid_train_validation_gap", float(worst["train_validation_gap"]), f"{float(worst['
                train_validation_gap']):.2f}"),
            ("maximum_gap_C", float(worst["C_full_precision"]), display_parameter(float(worst["
                C_full_precision"]))),
            ("maximum_gap_gamma", float(worst["gamma_full_precision"]), display_parameter(float(worst["
                gamma_full_precision"]))),
            ("selected_gamma_at_lower_grid_boundary", float(np.isclose(float(search.best_params_["
                svc__gamma"]), GAMMA_GRID.min())), "Yes" if np.isclose(float(search.best_params_["
                svc__gamma"]), GAMMA_GRID.min()) else "No"),
        ],
        columns=audit.columns,
    )
    audit = pd.concat([audit, extra_audit], ignore_index=True)
    pareto = build_pareto_table(grid)

    CSV_DIR.mkdir(parents=True, exist_ok=True)
    predictions.insert(1, "PatientID", frame["PatientID"])
    nested.to_csv(CSV_DIR / "rbf_nested_cv.csv", index=False, float_format="%.15g")
    predictions.to_csv(CSV_DIR / "rbf_oof_predictions.csv", index=False, float_format="%.15g")
    grid.to_csv(CSV_DIR / "rbf_grid_search.csv", index=False, float_format="%.15g")
    audit.to_csv(CSV_DIR / "rbf_solution_audit.csv", index=False, float_format="%.15g")
    support.to_csv(CSV_DIR / "rbf_support_vectors.csv", index=False, float_format="%.15g")
    comparison.to_csv(CSV_DIR / "linear_vs_rbf_comparison.csv", index=False, float_format="%.15g")
    pareto.to_csv(CSV_DIR / "svm_grid_pareto_frontier.csv", index=False, float_format="%.15g")
    pd.DataFrame([metrics]).to_csv(CSV_DIR / "rbf_nested_metrics.csv", index=False, float_format="%.15g")

    save_tables(nested, audit, comparison)
    save_grid_figure(
        grid,
        float(search.best_params_["svc__C"]),
        float(search.best_params_["svc__gamma"]),
    )
    save_roc_figure(predictions, nested)
    save_pareto_figure(pareto)

    print(f"Best C: {float(search.best_params_['svc__C']):.12g}")
    print(f"Best gamma: {float(search.best_params_['svc__gamma']):.12g}")
    print(f"Mean outer AUROC: {nested.outer_test_AUROC.mean():.6f}")
    print(comparison.to_string(index=False))
    print(audit.to_string(index=False))
    print("Pareto-optimal grid settings")
    print(
        pareto.loc[
            pareto["is_pareto_optimal"],
            ["model", "C_display_2dp", "gamma_display_2dp", "validation_AUROC_display_2dp", "
                gap_display_2dp"],
        ].to_string(index=False)
    )


if __name__ == "__main__":
    main()
```